Qualcomm Technologies, Inc.

SW6100/SW6100P Interfaces Guide – Linux Android Wear

User Guide

80-74841-87 Rev. AB

April 23, 2026

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-18 02:44:21 GMT
hyungchul.won@lge.com

About Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Building on our 40 years of technology leadership in creating era-defining breakthroughs, we deliver a broad portfolio of solutions built with our leading-edge AI, high-performance, low-power computing, and unrivaled connectivity. Our Snapdragon® platforms power extraordinary consumer experiences, and our Qualcomm Dragonwing™ products empower businesses and industries to scale to new heights. Together with our ecosystem partners, we enable next-generation digital transformation to enrich lives, improve businesses, and advance societies. At Qualcomm, we are engineering human progress.

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Contents

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyubinchul.won@lge.com

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

Tables

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-18 02:44:21 GMT
hyungchul.won@lge.com

Figures

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-18 02:44:21 GMT
hyungchul.won@lge.com

1 Interfaces documentation

The SW6100/SW6100P platform supports UART, SPI, I2C, I3C, and USB peripheral interfaces through its QUP v3 serial engine architecture. You can configure low-speed interfaces for sensor communication and Bluetooth connectivity, or enable high-speed USB for debugging and data transfer. Setup instructions, device tree configurations, and debugging procedures guide you through enabling each interface.

Section	Description
Interface overview	
Architecture Overview of peripheral interfaces	Learn about the software stack for peripheral interfaces.
QUPv3 QUP v3	Lists the transfer modes and access control customizations in QUPv3.
Getting started: Set up the device interface	
Identify interface status bootup logs Identify interface status	Obtain the logs, and lists the enabled interfaces.
Load Linux firmware Load firmware on QUP v3 serial engine	Loads the firmware of the required protocol.
Enable interface Enable required interfaces	Enable the required interface.
Verify interface status Verify interface status	Verify enabled interface.
UART	
Enable Zephyr Shell in AWM Enable Zephyr Shell in AWM	Enable Zephyr Shell in ambient mode wearable microcontroller.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-13 02:44:21 GMT hungchul.won@lge.com

Section	Description
Enable Zephyr Shell in SWM Enable Zephyr Shell in SWM (physical UART through GLINK)	Enable Zephyr Shell in sensor wearable microcontroller in physical UART through GLINK.
SPI	
Configure SPI SPI configuration	Configure SPI software driver and device tree nodes.
I2C	
Share TOP QUP and SSC QUP using eGPIO Share TOP QUP and SSC QUP using eGPIO	Share serial engine between APPS and SSC using eGPIO.
I3C	
I3C software I3C software	Overwrite the default GPIO configuration of a serial engine.
USB	
Configure USB boot loader Configure USB boot loader	Configure the USB boot loader using the QDTE tool.
Customize USB device Customize USB device	Customize USB for UVC, UAC, data role swap, and composition use cases.
Debug USB issues Debug USB issues	Troubleshoot USB issues.

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:41:21 GMT hyungchul.won@lge.com

2 Overview of peripheral interfaces

The SW6100/SW6100P provides 25 QUP v3 serial engines—11 for the application processor and 14 for the sensor low-power island (SLPI). Low-speed interfaces (UART, SPI, I2C, I3C) connect to sensors, displays, and Bluetooth devices, while the USB 2.0 controller handles high-speed peripheral communication. Interface-to-GPIO mapping tables help you identify available serial engines and plan peripheral connections.

- The low-speed I/O interfaces operate at a lower frequency than the high-speed interfaces in the SoC. The Qualcomm universal peripheral (QUP) v3 serial engine is a hardware core that supports the following low-speed peripheral interfaces:
  - Universal asynchronous receiver/transmitter (UART)
  - Serial peripheral interface (SPI)
  - Interintegrated circuit (I2C)
  - Improved interintegrated circuit (I3C)

  These low-speed interfaces communicate with low-speed peripherals, such as sensor interface devices, Bluetooth® wireless technology devices, display or touch interface devices.
- The high-speed I/O interface includes the following peripheral interface:
  - Universal serial bus (USB)

  The high-speed I/O interfaces are used to connect to high-speed devices, such as solid-state drives, network cards, external graphics cards.

Qualcomm Confidential – May Contain Trade Secrets
2026-05-11 02:44:21 GMT
hyungchul.ryu@lge.com

The following figure shows the software stack for peripheral interfaces.

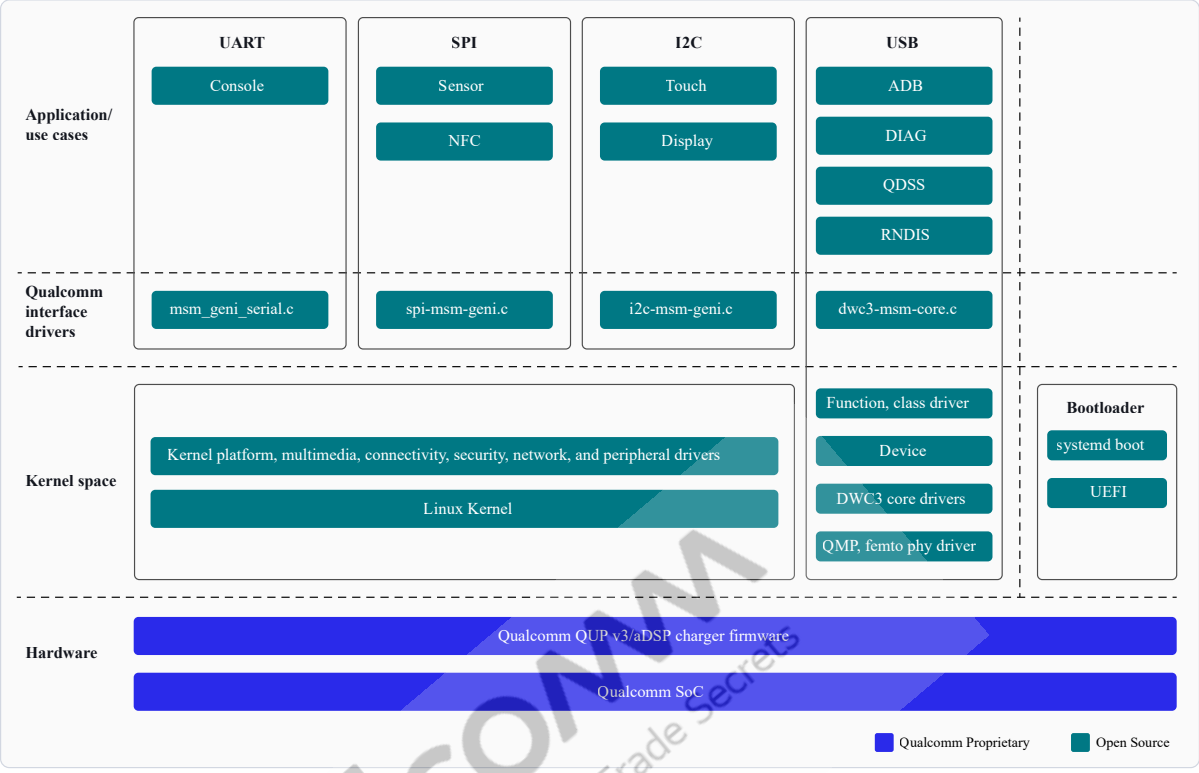


Figure 2-1 Peripheral interfaces software stack

QUP v3

Qualcomm uses QUP v3, a highly flexible and programmable hardware to support a wide range of serial interfaces. For more information about QUP v3, see the following resources:

- QUP v3 overview
- Supported transfer modes in QUP v3
- QUP v3 access control customization
- QUP v3 firmware status verification

NOTE The source code for boot and aDSP subsystems is available to licensed developers with authorized access.

This guide is your primary resource for information about enabling low-speed and high-speed I/O interfaces and making configuration changes.

2.1 SW6100/SW6100P interface overview

The following table lists the device interface features for SW6100.

Table 2-1 SW6100/SW6100P device interfaces

2xQUP v3 serial engine	
Serial engine instances	QUPV3_0

Table 2-1 SW6100/SW6100P device interfaces (cont.)

Application processor QUP v3 serial engine	11
LPI QUP v3 serial engine	14
1xUSB controller	
Controller address	0xa600000
Max speed	USB 2.0 high speed
HS/SS PHY power rails	■ L25 ■ L26

QUP v3 mapping to protocols and GPIOs in SW6100

SW6100/SW6100P has 25 QUP v3 serial engines. Of these, 11 QUP v3 serial engines are allocated for the application processor and 14 QUP v3 serial engines are allocated for the sensor low-power island (SLPI) on the application digital signal processor (aDSP).

Select only one protocol in a QUP v3 serial engine at a time. For example, simultaneous UART and I2C functions aren't supported. Each QUP v3 serial engine has up to seven lanes (I/O), which are numbered from 0 to 6.

NOTE The top-level QUP v3 serial engines are used as the application processor and boot image. The low-power island (LPI) QUP v3 serial engines are used for the sensor subsystem and Qualcomm® Trusted Execution Environment (TEE) use cases.

The following table lists the default QUP v3 mapping to protocols and GPIOs.

Table 2-2 SW6100/SW6100P QUP v3 serial engine protocol mapping

QUP_0 v3 serial engine	Protocols				QUP_0 lane to GPIO mapping				Use case
	I2C	I3C	UART	SPI	L0	L1	L2	L3	
SE0	Yes	Yes	Yes	Yes	0	1	2	3	[SPI] NFC_ESE_SPI
SE1	Yes	Yes	Yes	Yes	4	5	7	24	[I2C] NFC_CTRL_I2C
SE2	Yes	–	Yes	Yes	8	9	10	11	[SPI] QCC_UWB_SPI
					134	133	132	131	NO DEFAULT USE CAS
SE3	Yes	Yes	Yes	Yes	111	112	14	15	NO DEFAULT USE CAS
					8	9	10	11	[SPI] *QCC_UWB_SPI
SE4	Yes	–	Yes	Yes	16	17	18	19	[UART] DEBUG_UART
SE5	Yes	–	Yes	Yes	23	22	21	20	[I2C] *APPS_I2C
					33	35	39	38	NO DEFAULT USE CAS
SE6	Yes	–	Yes	Yes	28	29	117	118	[UART] HIGH_SPEED_U
SE7	Yes	–	Yes	Yes	23	22	21	20	NO DEFAULT USE CAS
					127	128	129	130	[SPI] * GPIO_TOUCH_S
SE8	Yes	–	Yes	Yes	72	40	14	15	[I2C] RSB_I2C
SE9	Yes	–	Yes	Yes	28	29	14	15	[I2C] TOUCH_I2C

Qualcomm
Confidential – May Contain Trade Secrets
2026-05-18 02:44:21 GMT
hungchul.won@lge.com

Table 2-2 SW6100/SW6100P QUP v3 serial engine protocol mapping (cont.)

QUP_0 v3 serial engine	Protocols				QUP_0 lane to GPIO mapping				Use case
	I2C	I3C	UART	SPI	L0	L1	L2	L3	
SE10	Yes	–	Yes	Yes	2	3	21	20	NO DEFAULT USE CAS
When marked with an asterisk (*), HW connections are there and the SE firmware includes the protocol by default, but it is not the software.									

The following table lists the default LPI QUP v3 mapping to protocols and GPIOs.

Table 2-3 SW6100/SW6100P LPI QUP v3 serial engine protocol mapping

LPI QUP v3 serial engine	Protocols				LPI QUP lane to GPIO mapping						
	I2C	I3C	UART	SPI	L0	L1	L2	L3	L4	L5	
SE0	Yes	Yes	–	–	103	104	–	–	–	–	[I3
SE1	Yes	Yes	–	–	105	106	–	–	–	–	[I2
SE2	Yes	Yes	Yes	Yes	107	108	109	110	111	112	[UA
SE3	Yes	Yes	–	–	111	112	–	–	–	–	[I3
SE4	Yes	–	Yes	Yes	113	114	115	116	121	122	[UA
SE5	Yes	Yes	Yes	–	117	118	117	118	–	–	[UA
SE6	Yes	Yes	Yes	–	119	120	119	120	–	–	NO
SE7	Yes	–	Yes	–	121	122	121	122	–	–	[I2
SE8	Yes	Yes	–	–	123	124	–	–	–	–	[I2
SE9	Yes	Yes	Yes	–	125	126	119	120	–	–	[UA
SE10	Yes	–	–	–	72	40	–	–	–	–	[I2
SE11	Yes	–	–	Yes	134	133	132	131	–	–	NO
SE12	Yes	–	–	–	28	29	–	–	–	–	[I2
SE13	Yes	–	–	Yes	127	128	129	130	–	–	NO
When marked with an asterisk (*), HW connections are there and the SE firmware includes the protocol by default, bu											

Qualcomm Confidential - May Contain Trade Secrets 2025-05-18 02:44:21 GMT hyungpaul.won@lge.com

3 Getting started: Set up device interface

You can use the SW6100/SW6100P platform to bring up device interfaces using a straightforward workflow. You learn how to collect boot logs, identify available interfaces, load QUPv3 firmware, enable required peripherals, and verify operation. This guidance helps you move from initial setup to a working interface environment with minimal effort.

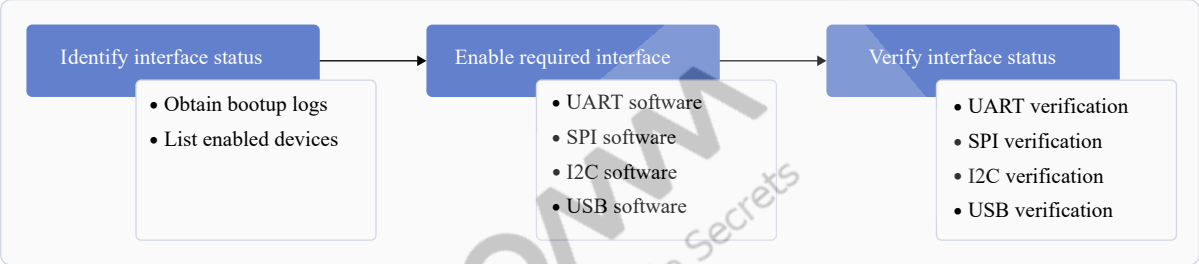


Figure 3-1 Device interface workflow

3.1 Identify interface status

The default interface status indicates the status of the interfaces during bootup. To identify the interface status, you must ensure that the interfaces are registered successfully and obtain the list of enabled interfaces.

3.1.1 Obtain the bootup logs

To obtain the bootup logs of the device, do the following:

1. Open the ADB shell.
2. Obtain the logs of the enabled interfaces by running the following command.

   ```
   dmesg
   ```

   Sample output:

```
[    0.000000][    T0] Kernel command line:
console=ttynull stack_depot_disable=on cgroup_disable=pressure
kasan.stacktrace=off kvm-arm.mode=protected bootconfig
android_arch_task_struct_size=512 kasan=off kpti=0
log_buf_len=256K swiotlb=0 loop.max_part=7 irqaffinity=0-1
cpufreq.default_governor=performance printk.console_no_auto_verbose=1
rcupdate.rcu_expedited=1 rcu_nocbs=0-4 kernel.panic_on_rcu_stall=1
lpm_levels.sleep_disabled=1 video=vfb:640x400,bpp=32,memsize=3072000
msm_rtb.filter=0x237 service_locator.enable=1 swiotlb=noforce
kpti=off cgroup.memory=nokmem,nosocket loop.max_part=7 bootconfig
androidboot.selinux=enforcing console=ttyMSM0,115200n8 earlycon
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:41:41 GMT hyungchul.won@lge.com

```
qcom_geni_serial.con_enabled=0 bootconfig  rootwait ro init=/init
silent_boot.mode=nonsilent
[    0.046117][    T1] Serial: AMBA PL011 UART driver
[    0.434424][   T12] a90000.serial: ttyMSM0 at MMIO 0xa90000 (irq =
140, base_baud = 0) is a MSM

[   11.412733] usbserial: USB Serial support registered for FTDI USB
Serial Device
[   11.523252] modprobe: Loading module /system/lib/modules/
usbserial.ko with args ''
[   12.096254] boot_kpi: M - DRIVER GENI_I2C Init
[   12.096291] i2c_geni aa4000.i2c: Bus frequency is set to 400000Hz.
[   12.288192] i2c_geni aa4000.i2c: I2C probed
```

3.1.2 List enabled interfaces on devices

To obtain the list of the enabled interfaces, do the following:

- For UART, run the following command.

  ```
  ls /dev/tty*
  ```

  Sample output:

  ```
  /dev/tty  /dev/ttyEUD0  /dev/ttyHS1  /dev/ttyMSM0  /dev/ttynull
  ```

- For I2C, run the following command.

  ```
  ls /dev/i2c*
  ```

  The following output is displayed.

  ```
  dev/i2c-0 /dev/i2c-1
  ```

- For SPI, run the following command.

  ```
  ls /dev/spi*
  ```

  The following output is displayed.

  ```
  spidev14.0
  ```

3.2 Load firmware on QUP v3 serial engine

The QUP v3 serial engine loads the firmware for the required protocol onto the serial engine. The configuration of protocols (I2C, SPI, I3C) and communication modes (FIFO, GSI), is done in a secure execution environment, such as Qualcomm TEE `devcfg`.

Default configuration

The default configuration for each serial engine, including the selected mode of data transmission and ownership, is in the `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c` file.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyunchul.won@lge.com

In the default configuration, Linux owns all the serial engines. The Qualcomm TEE manages all secure use cases.

```
const QUPv3_se_security_permissions_type qupv3_perms_wdp[] =
{
  /*   PeriphID,         ProtocolID,               Mode,  NsOwner,
bAllowFifo, bLoad, bModExcl  */
  { QUPV3_1_SE0, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_TZ,
FALSE,    TRUE,  TRUE  }, // NFC ESE SPI
  { QUPV3_1_SE1, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // NFC Ctrl I2C
  { QUPV3_1_SE2, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // QCC UWB SPI eCoRRAL_SKU1 (SKU1), WDP
  { QUPV3_1_SE3, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // QCC UWB SPI
  { QUPV3_1_SE4, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_FIFO, AC_HLOS,
TRUE,     FALSE, FALSE }, // DEBUG_UART
  { QUPV3_1_SE5, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_FIFO, AC_HLOS,      TRUE,
   TRUE,  FALSE }, // APPS I2C
  { QUPV3_1_SE6, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE },
  { QUPV3_1_SE7, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // eGPIO Touch SPI
// The dedicated SOCCP & DCP ownership usecase shall be taken care in XBL. No
need to mention the configs here
  /*{ QUPV3_1_SE8, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_DCP,
FALSE,    TRUE,  FALSE },*/ // DCP_UART
  { QUPV3_1_SE9, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE },//touch
  /*QUPV3_1_SE10*/
};
```

3.3 Enable required interfaces

To enable an interface, do the following:

- For UART, see the UART software section.
- For SPI, see the SPI software section.
- For I2C, see the I2C software section.
- For USB, see the USB software section.

3.4 Verify interface status

To verify the functioning of the different interfaces, do the following:

- For UART, see the Verify UART interface section.
- For SPI, see the SPI verification section.

Qualcomm Confidential - May Contain Trade Secrets 2020-05-18 02:44:21 GMT hyungchul.won@lge.com

- For I2C, see the Verify I2C interface section.
- For USB, connect the USB Type-C port and verify the enumerated log with the host PC.

  ```
  adb devices
  ```

  The following output is displayed.

  ```
  List of devices attached
  541eb4ba        device
  ```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

4 UART

The SW6100/SW6100P UART interface supports asynchronous serial communication for debug consoles, Bluetooth modules, and peripheral devices at baud rates up to 9.6 Mbps. You can configure 2-wire or 4-wire modes, enable Zephyr Shell for M55 processor debugging, and run loopback tests to verify connections. Device tree settings, pinctrl configurations, and Qualcomm TEE access control parameters control UART behavior across Linux, boot, and aDSP subsystems.

UART devices transmit data asynchronously. Hence, a clock signal doesn't synchronize the output of bits from the transmitting UART to the sampling of bits by the receiving UART. Instead of a clock signal, the transmitting UART adds start and stop bits to the data packet being transferred. These bits define the beginning and end of the data packet, so the receiving UART knows when to start reading the bits. When the receiving UART detects a start bit, it starts to read the incoming bits at a specific frequency known as the baud rate.

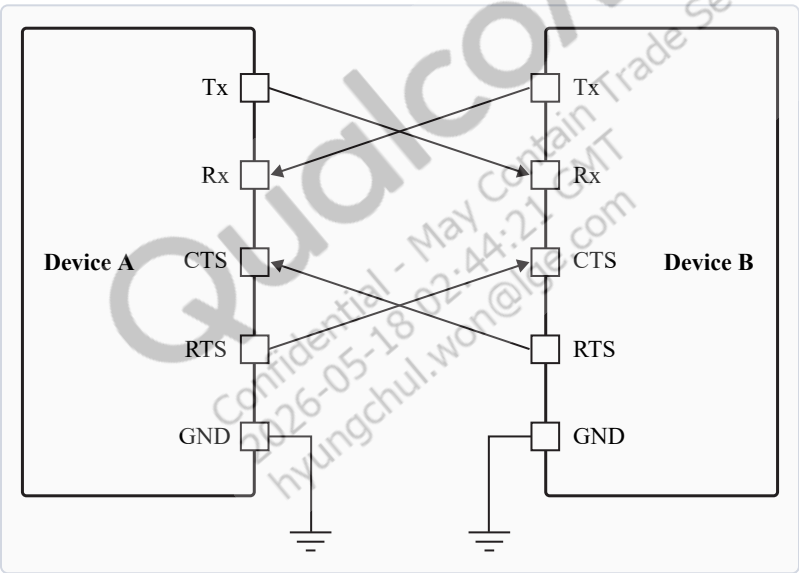


Figure 4-1 Data transfer between two UART devices

The parameters that determine successful transmission are as follows:

- Baud rate
- Start bit
- Stop bit
- Parity bit
- Data bits
- Flow control

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

The following figure shows a sample UART data packet.

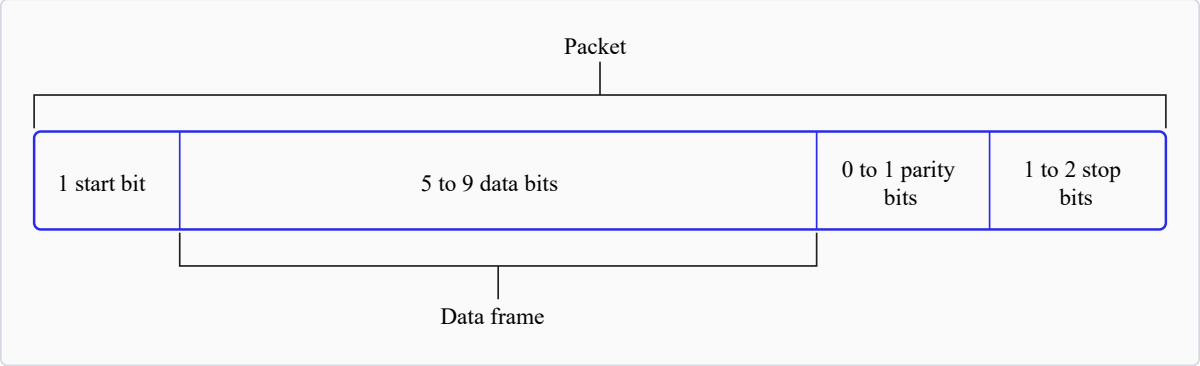


Figure 4-2 UART data packet

4.1 UART features

The following table describes the UART transfer modes for applications.

Table 4-1 UART transfer modes

Subsystem	Transfer mode	Description
Linux	▪ FIFO (low speed) ▪ CPU DMA (high speed)	▪ Supports baud rates from 6 Mbps up to 9.6 Mbps, based on the serial engine. ▪ FIFO mode transfers data between its Rx/Tx buffers and system memory. ▪ DMA mode transfers data between its Rx/Tx buffers and system memory. Better performance is achieved with high-speed UART drivers supporting higher baud rates and bigger data packages.
Boot	FIFO	▪ Rx/Tx 5 bits to 8 bits per character. ▪ Supports a maximum baud rate of 3.75 Mbps.
aDSP	FIFO	▪ Rx/Tx 5 bits to 8 bits per character. ▪ Supported baud rates: 9.6 Mbps.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

4.2 UART interface

The following table provides the paths of UART driver configurations for the different subsystems.

Table 4-2 UART interface: Linux

File type	Description
Device tree source	msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi
Pinctrl settings	msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi
Qualcomm TEE settings	trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Acc

Table 4-3 UART interface: Boot (UEFI-only)

File type	Description
QUP v3 serial engine configuration	■ Console UART:\boot_images\boot\QcomPkg\SocPkg\Aspen\Settings\UART\ UartSettings.c ■ HS UART: ▫ boot_images\boot\Settings\Platform\Aspen_Default\Core\Buses\qup_common\qupv3. ▫ boot_images\boot\Settings\Soc\Aspen\Core\Buses\qup_common\qupv3.dtsi
Qualcomm TEE settings	trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_A

Table 4-4 UART interface: aDSP/SLPI

File type	Description
QUP v3 serial engine configuration	adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\ssc-qupv3.dtsi
Firmware configuration settings	\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi

Table 4-5 UART interface: Arm® Cortex®-M55 processor

File type	Description
QUP v3 serial engine configuration	■ AWM ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_awm\core\buses\aspen-qupv3.dtsi ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_awm\core\buses\aspen-ssc-qupv3-se.dtsi

Qualcomm Confidential - May Contain Trade Secrets 2026-05-10 02:44:21 GMT hyungchul.won@lge.com

Table 4-5 UART interface: Arm® Cortex®-M55 processor (cont.)

File type	Description
	▪ SWM ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_swm\core\buses\aspen-qupv3.dtsi ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_swm\core\buses\aspen-ssc-qupv3-se.dtsi
Firmware configuration settings	▪ \adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi

4.2.1 UART APIs

UART APIs for the following subsystems are listed in this section.

- Linux:https://github.com/torvalds/linux/blob/master/include/linux/tty.h
- Boot: QcomPkg/Include/HSUart.h
- aDSP: adsp_proc/core/api/common/buses/uart.h
- M55 processor: core.wasp/buses/uart/zephyr/uart_qcom.c

4.3 UART software

This section provides information on the UART device tree configuration, and documentation for the device nodes.

Linux

For information about Kernel device instances, see Documentation/devicetree/bindings/serial/qcom,serial-geni-msm.yaml.

For information about the UART driver files, see drivers/tty/serial/msm_geni_serial.c.

For configuration settings of the serial engine, see msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi.

```
        uart6: serial@a98000 {
            compatible = "qcom,msm-geni-serial-hs";
            reg = <0xa98000 0x4000>;
            reg-names = "se_phys";
            interrupts = <GIC_SPI 362 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&gcc GCC_QUPV3_WRAP1_S6_CLK>;
            clock-names = "se-clk";
            interconnects = <&clk_virt MASTER_QUP_CORE_1 &clk_virt
SLAVE_QUP_CORE_1>,
                    <&gem_noc MASTER_APPSS_PROC &config_noc SLAVE_QUP_1>,
                    <&aggre_noc MASTER_QUP_1 &mc_virt SLAVE_EBI1>;
            interconnect-names = "qup-core",
                         "qup-config",
                         "qup-memory";
```

```
            pinctrl-0 = <&qupv3_se6_2uart_sleep>;
            pinctrl-1 = <&qupv3_se6_2uart_tx_active>,
<&qupv3_se6_2uart_rx_active>;
            pinctrl-2 = <&qupv3_se6_2uart_sleep>;
            pinctrl-3 = <&qupv3_se6_2uart_sleep>;
            pinctrl-names = "default",
                    "active",
                    "sleep",
                    "shutdown";
            dmas = <&gpi_dma1 0 6 QCOM_GPI_UART 64 0>,
                   <&gpi_dma1 1 6 QCOM_GPI_UART 64 0>;
            dma-names = "tx",
                    "rx";
            status = "disabled";
        };
```

For configuration settings of the serial engine GPIOs, see `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi`.

```
qupv3_se6_2uart_pins: qupv3_se6_2uart_pins {
            qupv3_se6_2uart_tx_active: qupv3_se6_2uart_tx_active {
                mux {
                    pins = "gpio117";
                    function = "qup0_se6_l2";
                };

                config {
                    pins = "gpio117";
                    drive-strength = <2>;
                    bias-disable;
                };
            };

            qupv3_se6_2uart_rx_active: qupv3_se6_2uart_rx_active {
                mux {
                    pins = "gpio118";
                    function = "qup0_se6_l3";
                };

                config {
                    pins = "gpio118";
                    drive-strength = <2>;
                    bias-disable;
                };
            };

            qupv3_se6_2uart_sleep: qupv3_se6_2uart_sleep {
                mux {
                    pins = "gpio117", "gpio118";
```

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungbul.won@lge.com

```
                    function = "gpio";
                };

                config {
                    pins = "gpio117", "gpio118";
                    drive-strength = <2>;
                    bias-pull-down;
                };
            };
        };
```

To enable the SE6 configuration as 2-wire UART, see `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna.dtsi`.

```
aliases {
        hsuart1 = &uart6;
}

&uart6 {
    qcom,auto-suspend-disable;
    status = "ok";
};
```

NOTE You must align the Qualcomm TEE configurations in the `QUPAC_Access.c` file to ensure that the GPIO/QUP v3 can be used. You can access the Qualcomm TEE images at `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\asper` Modify the required settings or see the default settings assigned for particular instances of the QUP v3

QUP v3 supports both 4-wire UART with flow-control enabled, and 2-wire UART without flow control enabled. The following example for Qualcomm TEE access, controls entry for both types of UART.

The default configuration enabled for SE6 as UART is as follows.

```
{ QUPV3_1_SE6, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_HLOS,       FALSE,
   TRUE,  FALSE }
```

Boot

Configure the QUP v3 serial engine to UART in boot using the `\boot_images\boot\QcomPkg\SocPkg\Aspen\Settings\UART\UartSettings.c` file.

```
UART_PROPERTIES devices =
{
   // MAIN_PORT
   0x00A90000,  // uart_base
   0x00AC0000,  // qup_common_base
   {
    {UART_GPIO_INVALID,{.gpio_config_value = UART_GPIO_INVALID,}},         //
qup_gpio_config[0] -CTS
        {UART_GPIO_INVALID,{.gpio_config_value =
UART_GPIO_INVALID,}},        //qup_gpio_config[1] - RFR
    {UART_TX_GPIO_NUM,
```

```
{{UART_TX_FUN_SELECT,UART_GPIO_OUT,UART_GPIO_PULL_UP,UART_GPIO_2MA_DRIVE,UART_
NO_STRONG_PULL,0}}},   //qup_gpio_config[2] - TX
    {UART_RX_GPIO_NUM,
{{UART_RX_FUN_SELECT,UART_GPIO_OUT,UART_GPIO_PULL_DOWN,UART_GPIO_2MA_DRIVE,UAR
T_NO_STRONG_PULL,0}}}, //qup_gpio_config[3] -RX
   },
   0,           // clock_id_index
   (void*)0,         // bus_clock_id
   "gcc_qupv3_wrap1_s4_clk",        //core_clock_id
   (void*)"gcc_qupv3_wrap1_s4_clk",     // core_clock_id
   0,        //base_freq
   0,          // irq number not used
   0,  //TCSR base
   0,  // TCSR offset
   0,  // TCSR value
   115200 //baud rate
};
```

aDSP

The firmware loads SSC QUP during the bootup sequence of the aDSP subsystem. Hence, the configuration file is present in the aDSP build at `\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi`.

The following configuration is a sample of SSC QUP SE2 loaded with the UART firmware in the FIFO mode.

```
 se2_cfg {
                offset              = /bits/ 32 <0x88000>;
                ibi_base            = /bits/ 32 <0x22120000>;
                protocol            = /bits/ 32 <SE_PROTOCOL_UART>;
                se_island_config    = /bits/ 16 <SNS_ISLAND_TYPE>;
                tre_list_size       = /bits/ 8 <1>;
                ibi_se_index        = /bits/ 8 <2>;
                se_mode             = /bits/ 8 <FIFO>;
                load_fw             = /bits/ 8 <1>;
                dfs_mode            = /bits/ 8 <0>;
            };
```

GPIO configuration: Each serial engine in the QUP common driver is configured with the default GPIO configuration per protocol. The QUP v3 common driver picks the GPIO configuration according to the protocol loaded in the serial engine from `\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\qup-ssc-pinctrl.dtsi`.

The default GPIO configuration can be overwritten as follows.

```
qup_ssc0_se2_uart_active: qup_ssc0_se2_uart_active{
        config = <&ssc_qupv3_se2_0 ssc_uart_active_cts_cfg>,
                 <&ssc_qupv3_se2_1 ssc_uart_active_rts_cfg>,
                 <&ssc_qupv3_se2_2 ssc_uart_active_tx_cfg>,
```

Qualcomm Confidential - May Contain Trade Secrets 2020-05-10 02:44:21 GMT hungchul.yoo@lge.com

```
                <&ssc_qupv3_se2_3 ssc_uart_active_rx_cfg>;
    };

    qup_ssc0_se2_uart_sleep: qup_ssc0_se2_uart_sleep{
        config = <&ssc_qupv3_se2_0 ssc_uart_sleep_cfg>,
                 <&ssc_qupv3_se2_1 ssc_uart_sleep_cfg>,
                 <&ssc_qupv3_se2_2 ssc_uart_sleep_cfg>,
                 <&ssc_qupv3_se2_3 ssc_uart_sleep_cfg>;
    };
```

TLMM_MAP is a macro to initialize the active and sleep state GPIO configurations. For example, sample usage of the TLMM_MAP macro.

```
TLMM_MAP (active state pull type, drive strength, sleep state pull type)
```

4.4 UART tools

This section provides information on various test tools and methods for the UART serial interface driver to confirm the UART data transfers.

Linux

For more details, see https://docs.kernel.org/admin-guide/serial-console.html.

4.5 Enable Zephyr Shell in AWM

This section describes how to enable Zephyr™ Shell in Ambient mode wearable microcontroller (AWM) and sensor wearable microcontroller (SWM) environments.

You can use the Zephyr Shell to interact directly with the Arm® Cortex®-M55 processor and independently test AWM and SWM use cases. By default, external release builds disable UART for AWM/SWM to conserve power, and prevent access to the Zephyr Shell in these builds.

To enable Zephyr Shell, follow these steps:

1. Assign **Debug UART** to Zephyr Shell.
   a. Edit the device tree source file: `wm_proc/core_ect/dt_settings/soc/aspen_lpai_awm/core/buses/aspen-ssc-qupv3-se.dtsi`.
   b. Replace the existing shell UART configuration.

```
chosen {
    zephyr,console = &ssc_qup0_se4;
    // Remove: qc,shell-uart = &ssc_qup0_se4;
    // Add: zephyr,shell-uart = &ssc_qup0_se4;
}
```

2. Update the Zephyr Shell configuration flags by adding the following flags to the `wm_proc/core_ect/config/prj_aspen_lpai_awm_board.conf` file.
   - `CONFIG_SERIAL=y`
   - `CONFIG_SHELL=y`
   - `CONFIG_SHELL_ASYNC_API=y`

- CONFIG_SHELL_BACKEND_SERIAL_ASYNC_RX_BUFFER_SIZE=32
- CONFIG_SHELL_BACKENDS=y
- CONFIG_LOG=y
- CONFIG_LOG_MODE_MINIMAL=n
- CONFIG_LOG_MODE_DEFERRED=y
- CONFIG_LOG_PRINTK=y

3. Push the updated AWM binaries to the device using the sideloading procedure.
4. Reboot device.
   a. Reboot using ADB commands or the power key.
   b. Verify shell access and log prints on the designated serial port.

4.6 Enable Zephyr Shell in SWM (physical UART through GLINK)

The Ambient mode wearable microcontroller (AWM) and sensor wearable microcontroller (SWM) environments communicate internally through the GLINK channel. To enable Zephyr™ Shell in SWM, follow these steps:

1. Disable **Debug UART** serial engine in AWM.
   a. Edit the `wm_proc/core_ect/dt_settings/soc/aspen_lpai_awm/core/buses/aspen-ssc-qupv3-se.dtsi` device tree file.
   b. Change the SW4 status.

```
enable-loopback = <0>;
enable_flow_ctrl = <0>;
// Change: status = "okay";
// To: status = "disabled";
```

2. Remove UART/Zephyr Shell Flags from the AWM Kconfig file.
   a. Edit the `wm_proc/core_ect/config/prj_aspen_lpai_awm_board.conf` file.
   b. Comment out or remove UART-related flags.

```
#CONFIG_QC_UART=y
```

3. Assign debug UART to Zephyr Shell in SWM.
   a. Edit the `wm_proc/core_ect/dt_settings/soc/aspen_lpai_swm/core/buses/aspen-ssc-qupv3-se.dtsi` device tree file.
   b. Add SE4 node and shell assignment.

```
&ssc_qup0_se4 {
    compatible = "qcom,qup-uart";
    current-speed = <4000000>;
    parity-mode = <0>;
    stop-bits = <0>;
    data-bits = <3>;
    enable-loopback = <0>;
    enable_flow_ctrl = <0>;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-05-10 02:44:21 GMT hyung-hwo@lge.com

```
        status = "okay";
    }
    aliases {
        uart0 = &ssc_qup0_se4;
    }
    chosen {
        zephyr,console = &ssc_qup0_se4;
        zephyr,shell-uart = &ssc_qup0_se4;
    }
```

4. Update Zephyr Shell Configuration Flags in SWM.
   a. Edit the `wm_proc/core_ect/config/prj_aspen_lpai_swm_board.conf` file.
   b. Update the following flags.
      - `CONFIG_QC_UART=y`
      - `CONFIG_SERIAL=y`
      - `CONFIG_SHELL=y`
      - `CONFIG_SHELL_ASYNC_API=y`
      - `CONFIG_SHELL_BACKEND_SERIAL_ASYNC_RX_BUFFER_SIZE=32`
      - `CONFIG_SHELL_BACKENDS=y`
      - `CONFIG_LOG=y`
      - `CONFIG_LOG_MODE_MINIMAL=n`
      - `CONFIG_LOG_MODE_DEFERRED=y`
      - `CONFIG_LOG_PRINTK=y`
5. Configure the GPII number.
   a. Edit the `wm_proc/core_ect/buses/settings/gpi/hal/aspen_lpai_swm/gpi_tgt.h` file
   b. Update the GPII number according to the serial engine usage.

      ```
      #define NUM_GPII 4 // Increase as per SE usage
      ```
6. Push the updated AWM and SWM binaries to the device using the sideloading procedure.
7. Reboot device.
   a. Reboot using ADB commands or the power key.
   b. Verify shell access and log prints on the designated serial port.

4.7 Enable UART in kernel

This section provides information on how to enable UART in the kernel.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT jyung-chol.won@qti.qualcomm.com

Linux

The following driver kernel configurations are required to support the UART interface.

- UART driver: `drivers/tty/serial/msm_geni_serial.c`
- Kernel `defconfig` file path: `common/arch/arm64/configs/defconfig`

The following kernel configurations must be enabled.

- `CONFIG_QCOM_GENI_SE=y`
- `CONFIG_SERIAL_QCOM_GENI=y`

To enable a serial node for the loopback validation, apply the following patch to the `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna.dtsi` file.

```
+
+&uart6 {
+    status = "ok";
+};
```

NOTE You should compile the kernel configuration and device tree changes. After compilation, you can load the images to the device to verify the interface. For information about interface verification, see the Verify UART interface section.

Boot/aDSP

For customizations, see the UART software section.

4.8 UART customization

For information about customizing UART software, see QUP v3 access control customization.

4.9 Verify UART interface

This section describes the validation procedure for the UART drivers, and the test results for the Qualcomm drivers.

Linux

To enable the UART nodes, do the following and compile the kernel configuration.

1. To change the UART status to `OK` and assign it to the specific UART node, edit the `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi` DTSI file.

```
uart0: serial@a80000 {
            compatible = "qcom,msm-geni-serial-hs";
            reg = <0xa80000 0x4000>;
            reg-names = "se_phys";
            interrupts = <GIC_SPI 353 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&gcc GCC_QUPV3_WRAP1_S0_CLK>;
            clock-names = "se-clk";
            interconnects = <&clk_virt MASTER_QUP_CORE_1 &clk_virt
```

Qualcomm Confidential – May Contain Trade Secrets 2025-05-18 02:41:21 GMT hyungdeul.won@lge.com

```
SLAVE_QUP_CORE_1>,
                    <&gem_noc MASTER_APPSS_PROC &config_noc SLAVE_QUP_1>,
                    <&aggre_noc MASTER_QUP_1 &mc_virt SLAVE_EBI1>;
            interconnect-names = "qup-core",
                         "qup-config",
                         "qup-memory";
            pinctrl-0 = <&qupv3_se0_default_cts>, <&qupv3_se0_default_rts>,
                    <&qupv3_se0_default_tx>, <&qupv3_se0_default_rx>;
            pinctrl-1 = <&qupv3_se0_cts>, <&qupv3_se0_rts>,
                    <&qupv3_se0_tx>, <&qupv3_se0_rx_active>;
            pinctrl-2 = <&qupv3_se0_cts>, <&qupv3_se0_rts>,
                    <&qupv3_se0_tx>;
            pinctrl-3 = <&qupv3_se0_default_cts>, <&qupv3_se0_default_rts>,
                    <&qupv3_se0_default_tx>, <&qupv3_se0_default_rx>;
            pinctrl-names = "default",
                    "active",
                    "sleep",
                    "shutdown";
            qcom,suspend-ignore-children;
        ++    status = "ok";
        };
```

2. To verify the UART driver, do the following:
   a. Open the ADB shell.
   b. Register the UART.

      ```
      ls /dev/ttyHS*
      ```

      The following is a sample output.

      ```
      ls /dev/ttyHS*
      /dev/ttyHS1
      ```

3. Enable the loop back.

   ```
   echo 3 >sys/devices/platform/soc/<UART node>/loopback
   ```

   For example,

   ```
   echo 3 > /sys/devices/platform/soc/ac0000.qcom,qupv3_1_geni_se/
   a98000.serial/loopback
   ```

   ```
   echo -1 > /sys/devices/platform/soc/99c000.qcom,qup_uart/power/
   autosuspend_delay_ms
   ```

The UART devices registered in the kernel are listed. The UART driver follows the test sequence to enable loopback. After enabling the UART node in the DUT, run the following commands to verify that the UART instance is enabled in the DTSI file.

NOTE Use the ADB shell to write and read the data for the UART loopback.

1. Open the ADB shell.
2. Transfer data with the `echo` command.

   ```
   echo "This Document Is Very Much Helpful" > /dev/ttyHS1
   ```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

3. Read data in the UART device node.

```
cat /dev/ttyHS1
```

4.10 Debug UART issues

This section provides information about enabling the debug logs in the UART software driver.

Linux

```
cat /sys/kernel/debug/ipc_logging/<<device id>>.serial_irqstatus/log_cont
cat /sys/kernel/debug/ipc_logging/<<device id>>.serial_new/log_cont
cat /sys/kernel/debug/ipc_logging/<<device id>>.serial_rx/log_cont
cat /sys/kernel/debug/ipc_logging/<<device id>>.uart_tx/log_cont
cat /sys/kernel/debug/ipc_logging/<<device id>>.serial_misc/log_cont
cat /sys/kernel/debug/ipc_logging/<<device id>>.serial_pwr/log_cont
```

4.11 Customize UART debugging in alternate serial engine

To enable debug UART and shell functionalities in a non-default serial engine, follow these steps:

1. aDSP changes (Hexagon Processor)
   a. Edit the `adsp_proc/core/dt_settings/soc/aspen/core/buses/qup_fw/fw_devcfg.dtsi` device tree file.
   b. Load the UART protocol and configurations for the intended serial engine (for example, SE4).

```
se4_cfg {
    offset = /bits/ 32 <0x90000>;
    protocol = /bits/ 32 <SE_PROTOCOL_UART>;
    // Additional parameters as needed
    status = "ok";
}
```

2. LPAI changes (Arm® Cortex®-M55 processor)

   Prerequisites:

   - Select the serial engine for UART based on use case and device availability.
   - Ensure the same serial engine isn't enabled in both Ambient mode wearable microcontroller (AWM) and sensor wearable microcontroller (SWM).

   Steps:

   a. Enable the serial engine node in device tree.
      i. Edit the `wm_proc/core_ect/dt_settings/soc/aspen_lpai_swm/core/buses/aspen-ssc-qupv3-se.dtsi` device tree file.
      ii. Define the serial engine node with protocol parameters.

Qualcomm Confidential – May Contain Trade Secrets 2020-05-18 02:44:21 GMT seungchul.won@lge.com

b. Update the `aspen-qupv3.dtsi`.

i. Edit the `wm_proc/core_ect/dt_settings/soc/aspen_lpai_swm/core/buses/aspen-qupv3.dtsi` device tree file.

ii. Ensure supported protocol includes UART.

iii. Update `gpii_list` with a unique number for each serial engine.

iv. Set status to `ok`.

c. Update shell configuration flags.

i. Edit the `wm_proc/core_ect/config/prj_aspen_lpai_swm_board.conf` file.

ii. Add UART and shell flags.

d. Configure GPII number.

e. Edit the `wm_proc/core_ect/buses/settings/gpi/hal/aspen_lpai_swm/gpi_tgt.h` file according to the serial engine usage.

f. Update NUM_GPII according to the serial engine usage.

3. Push the updated aDSP, AWM, and SWM binaries to the device using the sideloading procedure.

4. Reboot device.

a. Reboot using ADB commands or the power key.

b. Verify shell access and log prints on the designated serial port.

4.12 UART examples

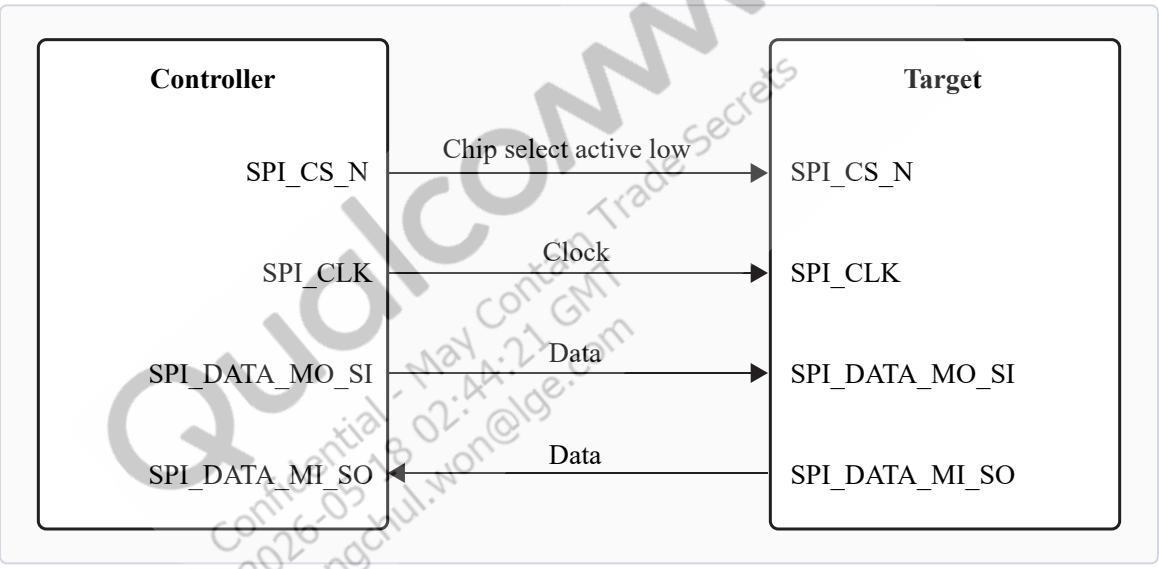
For information about the device tree reference, see the `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi` DTSI file.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:15 GMT hyungchul.won@lge.com

5 SPI

The SW6100/SW6100P serial peripheral interface (SPI) interface enables synchronous, full-duplex communication with NFC secure elements, UWB modules, and touch controllers at speeds up to 50 MHz. You can configure MOSI, MISO, CLK, and CS GPIO pins through device tree nodes and validate bus connectivity using spidev_test. FIFO, DMA, and GSI transfer modes are available depending on your subsystem and performance requirements.

The SPI is a synchronous serial data link that operates in full duplex mode. SPI is also referred to as a 4-wire serial bus.



Qualcomm Confidential - May Contain Trade Secrets 2026-03-18 02:44:21 GMT hyungchul.won@lge.com

Figure 5-1 SPI data flow

The SPI core supports the bidirectional SPI standard, point-to-point, and controller-target protocol. The SPI core uses four chip-select lines (*SPI_CS#_N*) to select target devices for communication. The following two data lines support the bidirectional data transfer.

- SPI_DATA_MO_SI: Controller data output, target data input.
- SPI_DATA_MI_SO: Controller data input, target data output.

Table 5-1 Data and control signals in SPI

Data signals	MOSI: Controller data output, target data input.
	MISO: Controller data input, target data output.
Control signals	SCLK: A clock generated by the controller and input to all targets.
	CS: Chip-select, a target is selected when the controller asserts its *CS_N* signal.

5.1 SPI features

This section explains the SPI serial engine transfer modes and the different scenarios where each mode is used. This section also describes the FIFO and DMA enabled in various subsystem SPI drivers.

Table 5-2 SPI transfer modes

Subsystem	Transfer mode	Description
Linux	■ FIFO (low-speed) ■ CPU DMA (high-speed) ■ GSI	Supports a maximum configuration speed of 50 MHz.
Boot	FIFO	■ Supports transfer rates up to 50 MHz. The host sets the SPI clock frequency nearest to the requested frequency. ■ 4 bits to 32 bits per word of transfer. ■ Maximum of 4 chip selects (CS) per bus. ■ GSI mode isn't supported on boot. ■ The driver executes in polling mode.
aDSP	■ Full duplex ■ Half duplex ■ Synchronous ■ Serial communication	■ There is no explicit communication framing, error checking, or defined data word length. ■ Communication is strictly at the raw bit level. ■ Supports transfer rates up to 50 MHz. The host sets the SPI clock frequency nearest to the requested frequency. ■ 4 bits to 32 bits per word of transfer. ■ Maximum of 4 chip selects (CS) per bus.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

5.2 SPI interface

This section provides information about the subsystem drivers, kernel device tree nodes, and related documentation.

Table 5-3 SPI interface: Linux

File type	Description
Device tree source	■ msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi ■ Documentation/devicetree/bindings/spi/qcom,spi-msm-geni.yaml
Pinctrl settings	/msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi
Qualcomm TEE settings	trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Acc

Table 5-4 SPI interface: Boot

File type	Description
QUP v3 serial engine configuration	QUP v3 serial engine: ■ boot_images\boot\Settings\Platform\Aspen_Default\Core\Buses\qup_common\qupv3.dt ■ boot_images\boot\Settings\Soc\Aspen\Core\Buses\qup_common\qupv3.dtsi
Qualcomm TEE settings	trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_A

Table 5-5 SPI interface: aDSP/SLPI/SDC

File type	Description
QUP v3 serial engine configuration	adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\ssc-qupv3.dtsi
Firmware configuration settings	\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi

Table 5-6 SPI interface: M55 processor

File type	Description
QUP v3 serial engine configuration	■ AWM □ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_awm\core\buses\aspen-qupv3.dtsi □ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_awm\core\buses\aspen-ssc-qupv3-se.dtsi

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungshul.won@lge.com

Table 5-6 SPI interface: M55 processor (cont.)

File type	Description
	▪ SWM ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_swm\core\buses\aspen-ssc-qupv3-se.dtsi ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_swm\core\buses\aspen-ssc-qupv3-se.dtsi
Firmware configuration settings	\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi

Table 5-7 SPI interface: Qualcomm TEE

File type	Description
QUP v3 serial engine configuration	▪ trustzone_images\core\settings\buses\spi\qupv3\config\aspen\tz\spi_devcfg_user. ▪ \trustzone_images\core\settings\buses\spi\qupv3\config\aspen\tz\spi_devcfg_user
Qualcomm TEE settings	trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_A

5.2.1 SPI APIs

SPI APIs for the following subsystems are listed in this section.

- Linux:
  - https://github.com/torvalds/linux/blob/master/include/uapi/linux/spi/spidev.h.
  - https://github.com/torvalds/linux/blob/master/include/linux/spi/spi.h.
- Boot: boot_images/boot/QcomPkg/Include/SpiApi.h
- aDSP/SDC: adsp_proc/core/api/common/buses/spi_api.h
- M55 processor:/core.wasp/buses/spi/zephyr/spi_qcom.c

5.3 SPI software

This section provides information on the SPI device tree configuration and documentation for the device nodes.

Linux

For device SPI details, see the msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi DTSI files.

For more information about kernel documentation specific to the SPI device nodes, see Documentation/devicetree/bindings/spi/qcom,spi-msm-geni.yaml.

```
spi0: spi@a80000 {
            compatible = "qcom,spi-geni";
            reg = <0xa80000 0x4000>;
            #address-cells = <1>;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:44:25 GMT hyungchun.woo@lge.com

```
            #size-cells = <0>;
            reg-names = "se_phys";
            interrupts = <GIC_SPI 353 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&gcc GCC_QUPV3_WRAP1_S0_CLK>;
            clock-names = "se-clk";
            interconnects = <&clk_virt MASTER_QUP_CORE_1 &clk_virt
SLAVE_QUP_CORE_1>,
                    <&gem_noc MASTER_APPSS_PROC &config_noc SLAVE_QUP_1>,
                    <&aggre_noc MASTER_QUP_1 &mc_virt SLAVE_EBI1>;
            interconnect-names = "qup-core",
                         "qup-config",
                         "qup-memory";
            pinctrl-0 = <&qupv3_se0_spi_mosi_active>,
<&qupv3_se0_spi_miso_active>,
                    <&qupv3_se0_spi_clk_active>, <&qupv3_se0_spi_cs_active>;
            pinctrl-1 = <&qupv3_se0_spi_sleep>;
            pinctrl-names = "default",
                    "sleep";
            dmas = <&gpi_dma1 0 0 QCOM_GPI_SPI 64 0>,
                   <&gpi_dma1 1 0 QCOM_GPI_SPI 64 0>;
            dma-names = "tx",
                    "rx";
            spi-max-frequency = <50000000>;
            status = "disabled";
        };
```

For kernel documentation specific to the GPIO `pinctrl` configuration, see `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi`.The corresponding configurations of the QUP v3 serial engine GPIOs are mapped in the `pinctrl.dtsi` file.

For example:

```
qupv3_se0_spi_pins: qupv3_se0_spi_pins {
            qupv3_se0_spi_miso_active: qupv3_se0_spi_miso_active {
                mux {
                    pins = "gpio0";
                    function = "qup0_se0_l0";
                };

                config {
                    pins = "gpio0";
                    drive-strength = <6>;
                    bias-disable;
                };
            };

            qupv3_se0_spi_mosi_active: qupv3_se0_spi_mosi_active {
                mux {
                    pins = "gpio1";
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:41:21 GMT hyungchul.yon@lge.com

```
            function = "qup0_se0_l1";
        };

        config {
            pins = "gpio1";
            drive-strength = <6>;
            bias-disable;
        };
    };

    qupv3_se0_spi_clk_active: qupv3_se0_spi_clk_active {
        mux {
            pins = "gpio2";
            function = "qup0_se0_l2";
        };

        config {
            pins = "gpio2";
            drive-strength = <6>;
            bias-disable;
        };
    };

    qupv3_se0_spi_cs_active: qupv3_se0_spi_cs_active {
        mux {
            pins = "gpio3";
            function = "qup0_se0_l3";
        };

        config {
            pins = "gpio3";
            drive-strength = <6>;
            bias-disable;
        };
    };

    qupv3_se0_spi_sleep: qupv3_se0_spi_sleep {
        mux {
            pins = "gpio0", "gpio1",
                "gpio2", "gpio3";
            function = "gpio";
        };

        config {
            pins = "gpio0", "gpio1",
                "gpio2", "gpio3";
            drive-strength = <2>;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-03-18 02:44:21 GMT hyungchul.yon@lge.com

```
                    bias-disable;
                };
            };
        };
```

Ensure that the protocol configuration for a particular serial engine uses the SPI protocol in the file at `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`. Modify the required settings if needed or see the default settings assigned for QUP v3 serial engine instances.

Following is the sample configuration for enabling the SPI.

```
{ QUPV3_1_SE2, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,       FALSE,
   TRUE,  FALSE },
{ QUPV3_1_SE3, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,       FALSE,
   TRUE,  FALSE },
```

Boot

Configure the QUP v3 settings according to the Qualcomm Linux chip product requirements in the `\boot_images\boot\Settings\Soc\Aspen\Core\Buses\qup_common\qupv3.dtsi` file.

NOTE To enable the required QUP v3 serial engine, update the QUP v3 wrapper node status to `okay`, and the respective serial engine node to `disabled` state.

The following sample nodes help to configure the QUP v3 serial engine in boot.

QUP wrapper node sample

```
    /* QUPv3_1  wrapper  instance */
    TOP_QUP_1{
        compatible = "qcom,qup-controller";

        qup_id                  = /bits/ 8 <(QUP_1)>;
        core_base_addr          = <QUPV3_1_CORE_BASE_ADDRESS>;
        common_base_addr        = <(QUPV3_1_CORE_COMMON_BASE_ADDRESS)>;
        se_wrapper_base_addr    = <(QUPV3_1_CORE_SE_BASE_ADDRESS)>;
        core_frequency          = <100000000>;
        qup_flags               = <(QUP_FLAGS_UNUSED)>;
        num_se                  = /bits/ 8 <11>;
        status                  = "disabled";
```

Sample node of serial engine instance

```
        /* qup1 SE0 Instance */
        TOP_QUP_1_SE_0 {
            core_offset                 = <0x00000000>;
            se_flags                    = <(USES_DDR_BUFFER |
USES_INTERNAL_DDR_MEM | ENABLE_RECOVER_ON_TIMEOUT | POLLED_MODE)>;
            se_index                    = /bits/ 8 <0>;
            FIFO_MODE                   = /bits/ 8 <1>;
            protocol_supported          = /bits/ 8 <(I2C_SUPPORTED |
SPI_SUPPORTED | UART_SUPPORTED)>;
```

Qualcomm Confidential - May Contain Trade Secrets 2020-07-18 02:44:21 GMT hyungchul.won@lge.com

```
            interface_supported         = /bits/ 8 <(CORE_IRQ)>;
            gpi_index                   = /bits/ 8 <0xFF>;
            num_gpiis                   = /bits/ 8 <1>;
            ring_size_multiplier        = /bits/ 8 <1>;
            core_irq                    = /bits/ 16 <0>;
            pdc_irq                     = /bits/ 16 <0>;
            gpio_int_num                = /bits/ 16 <0>;
            i2c_hub                     = /bits/ 8 <0>;
            i2c_mm                      = /bits/ 8 <0>;
            SE_EXCLUSIVE                = /bits/ 8 <1>; /* TO BE MANUALLY
UPDATED */
            se_clock                    = "gcc_qupv3_wrap1_s0_clk";

            pinctrl-names               = "i2c-default", "i2c-sleep",
"spi-default", "spi-sleep", "uart-default", "uart-sleep";
            pinctrl-0                   = <&top_qup1_se0_i2c_active>;
            pinctrl-1                   = <&top_qup1_se0_i2c_sleep>;
            pinctrl-2                   = <&top_qup1_se0_spi_active>;
            pinctrl-3                   = <&top_qup1_se0_spi_sleep>;
            pinctrl-4                   = <&top_qup1_se0_uart_active>;
            pinctrl-5                   = <&top_qup1_se0_uart_sleep>;
            status                      = "disabled";
        };
```

For `pinctrl` definitions, see the GPIO configuration file at `boot_images\boot\Settings\Soc\Aspen\Core\Buses\qup_common\qupv3-pinctrl.dtsi`.

NOTE Verify Qualcomm TEE settings before changing the unified extensible firmware interface (UEFI) configuration. Ensure that the serial engine node is in FIFO_MODE and accessible from the application processor. Verify that the loaded protocol is according to the requirement.

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:41:21 GMT hyungchul.won@lge.com

aDSP/SDC

Firmware loading with SSC QUP is performed during the bootup sequence of the aDSP subsystem. The configuration file is present in the aDSP build at `\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi`.

The following configuration is a sample of the SSC QUP SE6 loaded with the SPI firmware.

```
se6_cfg {
            offset              = /bits/ 32 <0x98000>;
            ibi_base            = /bits/ 32 <0x22150000>;
            protocol            = /bits/ 32 <SE_PROTOCOL_SPI>;
            se_island_config    = /bits/ 16 <SNS_ISLAND_TYPE>;
            tre_list_size       = /bits/ 8 <1>;
            ibi_se_index        = /bits/ 8 <5>;
            se_mode             = /bits/ 8 <GSI>;
            load_fw             = /bits/ 8 <1>;
```

```
            dfs_mode              = /bits/ 8 <1>;
        };
```

GPIO configuration: Each serial engine in the QUP common driver is configured with the default GPIO configuration. The GPIO configuration is picked up by the QUP common driver according to the protocol loaded for the serial engine at `\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\qup-ssc-pinctrl.dtsi`.

The default GPIO configuration can be overwritten as follows.

```
qup_ssc0_se2_spi_active: qup_ssc0_se2_spi_active{
    config = <&ssc_qupv3_se2_0 ssc_spi_active_miso_cfg>,
             <&ssc_qupv3_se2_1 ssc_spi_active_mosi_cfg>,
             <&ssc_qupv3_se2_2 ssc_spi_active_clk_cfg>,
             <&ssc_qupv3_se2_3 ssc_spi_active_cs_cfg>;
};

qup_ssc0_se2_spi_sleep: qup_ssc0_se2_spi_sleep{
    config = <&ssc_qupv3_se2_0 ssc_spi_sleep_cfg>,
             <&ssc_qupv3_se2_1 ssc_spi_sleep_cfg>,
             <&ssc_qupv3_se2_2 ssc_spi_sleep_cfg>,
             <&ssc_qupv3_se2_3 ssc_spi_sleep_cfg>;
};
```

TLMM_MAP is a macro to initialize the active and sleep state GPIO configurations. For example, sample usage of the TLMM_MAP macro.

```
TLMM_MAP (active state pull type, drive strength, sleep state pull type)
```

Qualcomm TEE

The QUP v3 access configuration settings are at `\trustzone_images\core\settings\buses\spi\qupv3\config\aspen\tz\spi_devcfg_user.h`.

```
#define TZ_USE_SPI_15 //FP sensor
```

The `TZ_USE_SPI_<num>` number is based on the serial number of the serial engine (starting from one). For example, if there are two QUPs: QUPV3_0 with seven serial engines and QUPV3_1 with eight serial engines, the user must enable QUPV3_2_SE2. The macro should be TZ_USE_SPI_9.

GPIO configuration: In the following example, drive strength and pull are configured according to the PIN starting index from MISO at `trustzone_images\core\settings\buses\spi\qupv3\config\aspen\tz\spi_devcfg_user.c`.

```
ifdef TZ_USE_SPI_1
spi_plat_device_config_user spi_device_user_config_qup1_se0 =
{
    {2,2,2,2,-1,-1,-1},   //.drive_strength
    {1,1,0,1,-1,-1,-1},   //.pull
    0x08,                 //.gpii_idx
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT (yungchul.kim@lge.com)

```
    FALSE,                //.tpm (TRUE/FALSE)
    FALSE,                //.miso_sampling_ctrl_set
    0,                     //.mode_select  not supported for TZ
    0,                     //.flags not supported for TZ
};
```

Access control permission to the Qualcomm TEE subsystem is configured in the QUPAC policy at `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`.

```
const QUPv3_se_security_permissions_type qupv3_perms_wdp[] =
{
  /*   PeriphID,         ProtocolID,               Mode,  NsOwner,
bAllowFifo, bLoad, bModExcl  */
  { QUPV3_1_SE0, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_TZ,
FALSE,    TRUE,  TRUE  }, // NFC ESE SPI
  { QUPV3_1_SE1, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // NFC Ctrl I2C
  { QUPV3_1_SE2, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // QCC UWB SPI eCoRRAL_SKU1 (SKU1), WDP
  { QUPV3_1_SE3, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // QCC UWB SPI
  { QUPV3_1_SE4, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_FIFO, AC_HLOS,
TRUE,     FALSE, FALSE }, // DEBUG_UART
  { QUPV3_1_SE5, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_FIFO, AC_HLOS,      TRUE,
   TRUE,  FALSE }, // APPS I2C
  { QUPV3_1_SE6, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE },
  { QUPV3_1_SE7, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // eGPIO Touch SPI
// The dedicated SOCCP & DCP ownership usecase shall be taken care in XBL. No
need to mention the configs here
  /*{ QUPV3_1_SE8, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_DCP,
FALSE,    TRUE,  FALSE },*/ // DCP_UART
  { QUPV3_1_SE9, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE },//touch
  /*QUPV3_1_SE10*/

};
```

5.4 SPI bringup

This section describes how to enable the Qualcomm Linux SPI drivers.

Linux

To verify SPI communication with the device, kernel configuration must be enabled. The `CONFIG_SPI_SPIDEV=m` setting is enabled in the corresponding `<chipset> defconfig` file. Enable the specific QUP v3 SPI serial engine instance in the kernel device tree.

5.5 SPI configuration

This section provides information about the SPI software driver kernel configuration and device tree node changes.

Linux

The following driver kernel configurations are required to support the SPI interface.

- Driver source file at `drivers/spi/spi-msm-geni.c`
- Kernel `defconfig` file at `common/arch/arm64/configs/defconfig`

The following kernel configurations must be enabled.

- `CONFIG_QCOM_GENI_SE=y`
- `CONFIG_SPI_QCOM_GENI=m`
- `CONFIG_SPI_SPIDEV=m` to configure user space applications
- `CONFIG_QCOM_GPI_DMA=m` to enable GSI support

```
diff --git a/configs/vienna_perf.bzl b/configs/vienna_perf.bzl
index 03c0c2d0..68814b1 100644
--- a/configs/vienna_perf.bzl
+++ b/configs/vienna_perf.bzl
@@ -198,6 +198,7 @@
     "CONFIG_SDW_GPUCC_VIENNA": "m",
     "CONFIG_SERIAL_MSM_GENI": "m",
     "CONFIG_SPI_MSM_GENI": "m",
+    "CONFIG_SPI_SPIDEV": "m",
     "CONFIG_SPMI_MSM_PMIC_ARB": "m",
     "CONFIG_SPMI_MSM_PMIC_ARB_DEBUG": "m",
     "CONFIG_SPS": "m",
```

To enable the SPI node for the loopback validation, apply the following patch to the `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi` file.

```
+
+&spi14 {
+        status = "ok";
+        spidev@0 {
+                compatible = "spidev";
+                spi-max-frequency = <50000000>;
+                reg = <0>;
+        };
+};

++ drivers/spi/spi-msm-geni.c
-static unsigned bufsiz = 4096;
+static unsigned bufsiz = 35000;
```

Qualcomm Confidential – May Contain Trade Secrets 2020-05-18 02:14:21 GMT hyungsul.won@lge.com

```
module_param(bufsiz, uint, S_IRUGO);
MODULE_PARM_DESC(bufsiz, "data bytes in biggest supported SPI message");

@@ -742,6 +742,7 @@
     { .compatible = "semtech,sx1301", .data = &spidev_of_check },
     { .compatible = "silabs,em3581", .data = &spidev_of_check },
     { .compatible = "silabs,si3210", .data = &spidev_of_check },
+    { .compatible = "spidev"},
{},
 };
 MODULE_DEVICE_TABLE(of, spidev_dt_ids);
```

NOTE You should compile the kernel configuration and device tree changes. After the kernel is compiled, you can load the images to the device to verify the interface. For information about interface verification, see the SPI verification section.

5.6 SPI verification

This section describes the validation procedure and test results for the SPI drivers.

Linux

To cross-compile the SPI tools, do the following.

1. Install cross compiler.

   ```
   sudo apt-get install gcc-aarch64-linux-gnu
   ```

2. Set up the environment for cross-compilation by running the following command.

   ```
   export ARCH=arm64
   export CROSS_COMPILE=aarch64-linux-gnu-
   ```

3. Compile the tool by running the following command.

   ```
   make
   ```

Verify SPI device

Verify the driver by checking for `dev` node (`/dev/spidev1.0`) in the ADB shell.

1. To verify the SPI driver, do the following:
   a. Open the ADB shell.
   b. Mount the file system.

      ```
      mount -o remount,rw /usr
      ```

   c. Transfer files using SCP or similar tools.

      For example, `scp spidev_test root@10.92.175.138:/bin`.

   d. Assign permission to execute.

      ```
      chmod 777 spidev_test
      ```

Qualcomm Confidential – May Contain Trade Secrets 2026-05-10 02:44:21 GMT hyungchul.woo@lge.com

2. Verify an SPI device. Command format. `/spidev_test -D /dev/<spidev_node>`.

```
./spidev_test -D /dev/spidev1.0
./spidev_test -D /dev/spidev3.0
```

The following output is displayed.

```
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
```

Run the following command for information about usage.

```
./spidev_test -help
```

The following output is displayed.

```
-D --device   device to use (default /dev/spidev1.1)
-s --speed    max speed (Hz)
-d --delay    delay (usec)
-b --bpw      bits per word
-i --input    input data from a file (e.g. "test.bin")
-o --output   output data to a file (e.g. "results.bin")
-l --loop     loopback
-H --cpha     clock phase
-O --cpol     clock polarity
-L --lsb      least significant bit first
-C --cs-high  chip select active high
-3 --3wire    SI/SO signals shared
-v --verbose  Verbose (show tx buffer)
-p  Send data (e.g. "1234\xde\xad")
-N --no-cs    no chip select
  -R --ready    slave pulls low to pause
  -2 --dual     dual transfer
  -4 --quad     quad transfer
  -8 --octal    octal transfer
  -S --size     transfer size
  -I --iter     iterations
```

5.7 SPI debugging

This section describes the default logging method of the SPI software driver to enable logging the SPI transfer failures.

Linux

The SPI driver logs are enabled through a dynamic debugging method. Enable `CONFIG_DYNAMIC_DEBUG`in `common/arch/arm64/configs/defconfig` to support the dynamic debugging of the kernel drivers.

To enable and view the SPI driver logs in the kernel logs (`dmesg`), run the following command.

```
cat /sys/kernel/debug/ipc_logging/a88000.spi-a/log_cont
cat /sys/kernel/debug/ipc_logging/a88000.spi-a_tx_rx/log_cont
cat /sys/kernel/debug/ipc_logging/spi2.0/log_cont

mount -t debugfs none /sys/kernel/debug
echo -n "file spi-msm-geni.c +p" > /sys/kernel/debug/dynamic_debug/control
```

QUALCOMM Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyeongchul.won@lge.com

```
echo -n "file spidev.c +p" > /sys/kernel/debug/dynamic_debug/control
echo -n "file msm_gpi.c +p" > /sys/kernel/debug/dynamic_debug/control
```

5.8 SPI examples

For information about the upstream device tree reference, see the `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi` DTSI file.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

6 I2C

The SW6100/SW6100P interintegrated circuit (I2C) interface connects touch controllers, sensors, and display devices using a 2-wire bidirectional bus with support for standard (100 kHz), fast (400 kHz), and fast-mode plus (1 MHz) speeds. You can share I2C buses between the application processor and sensor subsystem using eGPIO configuration. Tools like i2cdetect and i2cdump help you scan for devices and debug communication issues.

I2C is a bidirectional 2-wire bus for an efficient inter-IC control bus developed by Philips in the 1980s. Every device on the bus has its own unique address (registered with the I2C general body headed by Philips). The I2C core supports a multicontroller mode and 10-bit target address and 10-bit extendable address. For more information about I2C, see https://www.i2c-bus.org/fileadmin/ftp/i2c_bus_specification_1995.pdf.

I2C communication sequence overview

The following figure shows the communication sequence between the controller and targets in I2C.

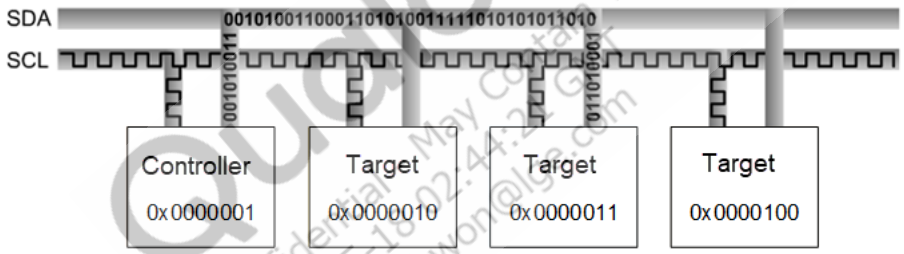


Figure 6-1 I2C controller and target communication sequence

For example, no device can use 1111-0XX listed in the I2C specification and high-speed mode with 3.4 MHz clock frequency.

Following are the I2C modes and supported speeds.

- Standard mode: 100 kbps
- Fast mode: 400 kbps
- Fast mode plus: 1 Mbps

The maximum supported bandwidth is 1 MHz.

I2C data packet format

The controller sends a 7-bit or 10-bit address as shown in the following figure. Along with the address, a 1-bit read/write indicating the type of operation is also sent. Data is transferred in sequences of 8 bits

Qualcomm Confidential - May Contain Trade Secrets 2020-05-18 02:44:22 GMT hyungchul.won@lge.com

placed on an SDA line. For every byte transferred, the device receiving the data sends back an ACK bit (totaling nine clock pulses).

- ACK bit LOW: receives the data and is ready to accept the next byte.
- ACK bit HIGH: receives the data and can't accept further data. The controller then terminates the transmission with the STOP sequence.

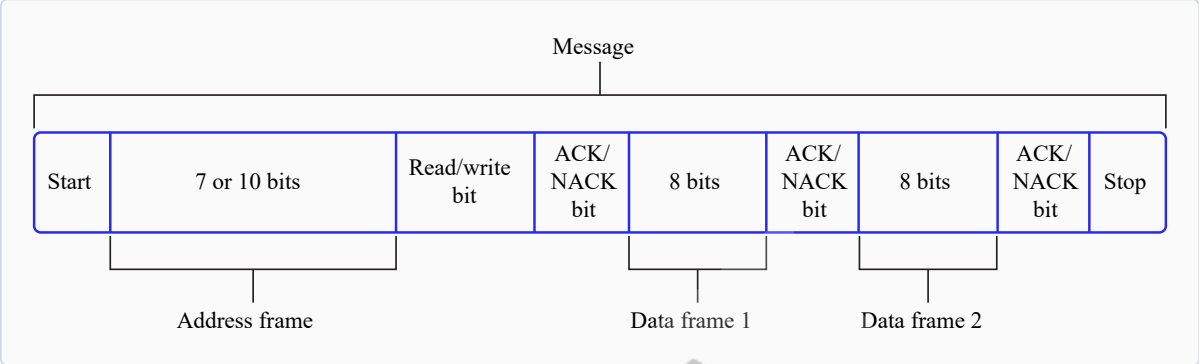
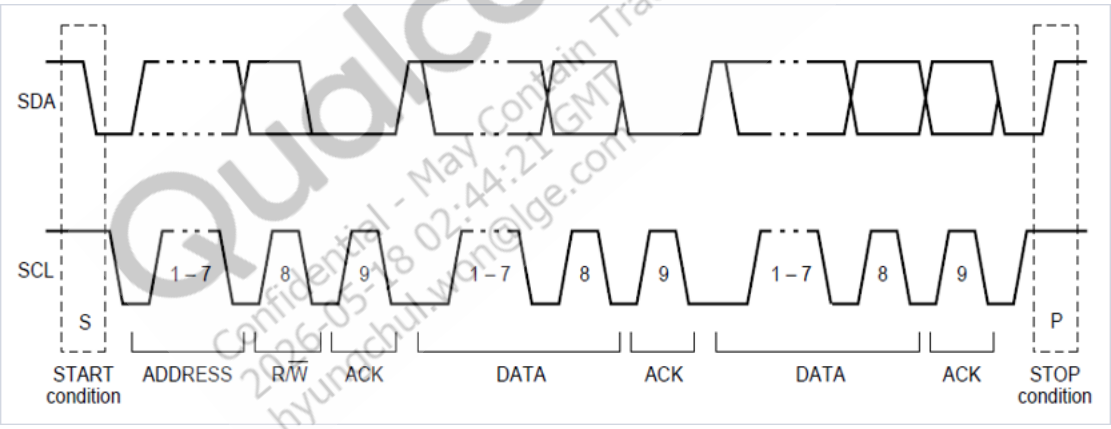


Figure 6-2 I2C data packet

I2C sequences

When the SCL is high, the SDA must remain stable and can't change. Only when the clock line is low can the data line change. However, there are two exceptions: START and STOP sequences.



Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyunchul.won@lge.com

6.1 I2C features

This section explains the I2C serial engine transfer modes and the different scenarios where each mode is used. The following table lists the transfer modes enabled in the various I2C subsystem drivers.

Table 6-1 I2C transfer modes

Subsystem	Transfer mode	Description
Linux	▪ FIFO (low speed) ▪ CPU DMA (high speed) ▪ GSI	▪ Supports 100 kHz, 400 kHz, and 1000 kHz bus speeds. ▪ Supports 7-bit target address according to the I2C specification.

Table 6-1 I2C transfer modes (cont.)

Subsystem	Transfer mode	Description
Boot	FIFO	■ Supports 100 kHz, 400 kHz, and 1000 kHz bus speeds. ■ Supports 7-bit target address according to the I2C specification.
aDSP/Qualcomm TEE/SDC	FIFO	

6.2 I2C interface

This section provides information about the subsystem driver, kernel device tree nodes, and related documentation.

Table 6-2 I2C interface: Linux

File type	Description
Device tree source	`msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi`
`Pinctrl` settings	`msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi`
Qualcomm TEE settings	`\trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`

Table 6-3 I2C interface: Boot

File type	Description
QUP v3 serial engine configuration	■ `boot_images\boot\Settings\Platform\Aspen_Default\Core\Buses\qup_common\qupv3.dtsi`QUP v3 serial engine: ■ `boot_images\boot\Settings\Soc\Aspen\Core\Buses\qup_common\qupv3.dtsi`
Qualcomm TEE settings	■ `\trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`

Table 6-4 I2C interface: aDSP/SLPI/SDC

File type	Description
QUP v3 serial engine configuration	`adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\ssc-qupv3.dtsi`
Firmware configuration settings	`\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi`

Table 6-5 I2C interface: M55 processor

File type	Description
QUP v3 serial engine configuration	■ AWM □ `\wm_proc\core_ect\dt_settings\soc\aspen_lpai_awm\core\buses\aspen-qupv3.dtsi` □ `\wm_proc\core_ect\dt_settings\soc\aspen_lpai_awm\core\buses\aspen-ssc-qupv3-se.dtsi`

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:44:11 GMT hyungshul.won@lge.com

Table 6-5 I2C interface: M55 processor (cont.)

File type	Description
	■ SWM ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_swm\core\buses\aspen-ssc-qupv3-se.dtsi ▫ \wm_proc\core_ect\dt_settings\soc\aspen_lpai_swm\core\buses\aspen-ssc-qupv3-se.dtsi
Firmware configuration settings	■ \adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi

Table 6-6 I2C interface: Qualcomm TEE

File type	Description
QUP v3 serial engine configuration	■ \trustzone_images\core\settings\buses\i2c\qupv3\config\aspen\tz\i2c_devcfg_user.c ■ \trustzone_images\core\settings\buses\i2c\qupv3\config\aspen\tz\i2c_devcfg_user.h.
Qualcomm TEE settings	■ \trustzone_images\core\settings\buses\qup_accesscontrol \qupv3\config\aspen\QUPAC_Access.c

6.2.1 I2C APIs

I2C APIs for the following subsystems are listed in this section.

- Linux:
  - https://github.com/torvalds/linux/blob/master/include/linux/i2c.h.
  - https://github.com/torvalds/linux/blob/master/include/linux/i2c-dev.h.
- Boot: boot_images/boot/QcomPkg/Include/i2c_api.h
- aDSP/SDC/SLPI: adsp_proc/core/api/common/buses/i2c_api.h
- M55 processor: core.wasp/buses/i2c/zephyr/i2c_qcom.c
- Qualcomm TEE: trustzone_images/core/buses/api/i2c/qupv3/i2c_api.h

6.3 I2C software

This section provides information on the I2C device tree configuration, and documentation for the device nodes.

Linux

For the configuration settings file, see the msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi DTSI file. For more details, see the i2c-geni-qcom.yaml file at /

Documentation/devicetree/bindings/i2c/qcom%2Ci2c-msm-geni.yaml, and the I2C driver at the drivers/i2c/busses/i2c-msm-geni.c files.

```
        i2c9: i2c@aa4000 {
            compatible = "qcom,i2c-geni";
            reg = <0xaa4000 0x4000>;
            #address-cells = <1>;
            #size-cells = <0>;
            interrupts = <GIC_SPI 427 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&gcc GCC_QUPV3_WRAP1_S9_CLK>;
            clock-names = "se-clk";
            interconnects = <&clk_virt MASTER_QUP_CORE_1 &clk_virt
SLAVE_QUP_CORE_1>,
                    <&gem_noc MASTER_APPSS_PROC &config_noc SLAVE_QUP_1>,
                    <&aggre_noc MASTER_QUP_1 &mc_virt SLAVE_EBI1>;
            interconnect-names = "qup-core",
                         "qup-config",
                         "qup-memory";
            pinctrl-0 = <&qupv3_se9_i2c_sda_active>,
<&qupv3_se9_i2c_scl_active>;
            pinctrl-1 = <&qupv3_se9_i2c_sleep>;
            pinctrl-names = "default",
                    "sleep";
            dmas = <&gpi_dma1 0 9 QCOM_GPI_I2C 64 0>,
                   <&gpi_dma1 1 9 QCOM_GPI_I2C 64 0>;
            dma-names = "tx",
                    "rx";
            status = "disabled";
        };
```

For kernel documentation specific to the GPIO pinctrl configuration, see the following files.

- /msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi
- /Documentation/devicetree/bindings/i2c/qcom,i2c-msm-geni.yaml

The corresponding configurations of the QUP v3 serial engine GPIOs are present and mapped in the msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi file.

```
        qupv3_se9_i2c_pins: qupv3_se9_i2c_pins {
            qupv3_se9_i2c_sda_active: qupv3_se9_i2c_sda_active {
                mux {
                    pins = "gpio28";
                    function = "qup0_se9_l0";
                };

                config {
                    pins = "gpio28";
                    drive-strength = <2>;
                    bias-pull-up;
                    qcom,i2c_pull;
                };
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-13 02:44:21 GMT

```
            };

            qupv3_se9_i2c_scl_active: qupv3_se9_i2c_scl_active {
                mux {
                    pins = "gpio29";
                    function = "qup0_se9_l1";
                };

                config {
                    pins = "gpio29";
                    drive-strength = <2>;
                    bias-pull-up;
                    qcom,i2c_pull;
                };
            };

            qupv3_se9_i2c_sleep: qupv3_se9_i2c_sleep {
                mux {
                    pins = "gpio28", "gpio29";
                    function = "gpio";
                };

                config {
                    pins = "gpio28", "gpio29";
                    drive-strength = <2>;
                };
            };
        };
```

In the `QUPAC_Access.c` file, ensure that the serial engine configuration for the specified protocol is for the I2C protocol. Qualcomm TEE build: `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`. Modify the required settings or refer to the default settings assigned for the QUP v3 serial engine instances. The following sample configuration is enabled by default in I2C.

```
{ QUPV3_1_SE9, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,       FALSE,
   TRUE,  FALSE }
```

Boot

1. Configure I2C in the UEFI. The configuration files can be accessed from the following locations.
   - QUP v3 serial engine: `boot_images\boot\Settings\Soc\Aspen\Core\Buses\qup_common\qupv3.dtsi`
   - Qualcomm TEE settings:`\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`

   NOTE For configuration settings in boot, see Boot.

2. Enable the I2C protocol in UEFI at `/QcomPkg/SocPkg/<chipset>/LAA/Core.fdf`.

   ```
   -#INF QcomPkg/Drivers/I2CDxe/I2CDxe.inf
   +INF QcomPkg/Drivers/I2CDxe/I2CDxe.inf
   ```

3. The application enables the I2C interface, then performs read and write operations through it. For information about I2C function usage, see `boot_images/QcomPkg/QcomTestPkg/I2CApp/I2Ceeprom.c`.
4. Add `i2c_open->i2c_read/i2c_write->i2c_close` sequentially in code.
5. Ensure that the `GpiDxe.inf` and `I2C.efi` files are loaded by verifying the device/UEFI bootup logs before you call `I2c_open`.

aDSP/SDC

Firmware loading with SSC QUP is performed during the bootup sequence of the aDSP subsystem. The configuration files are present in the aDSP build at `\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi`.

The following configuration is a sample of SSC QUP SE7 settings loaded with the I2C firmware.

```
se7_cfg {
     offset              = /bits/ 32 <0x9C000>;
     ibi_base            = /bits/ 32 <0x0>;
     protocol            = /bits/ 32 <SE_PROTOCOL_I2C>;
     se_island_config    = /bits/ 16 <SNS_ISLAND_TYPE>;
     tre_list_size       = /bits/ 8 <1>;
     ibi_se_index        = /bits/ 8 <0xFF>;
     se_mode             = /bits/ 8 <GSI>;
     load_fw             = /bits/ 8 <1>;
     dfs_mode            = /bits/ 8 <1>;
 };
```

GPIO configuration: Each serial engine in the QUP v3 common driver is configured with the default GPIO configuration. The GPIO configuration is picked up by the QUP v3 common driver according to the protocol loaded in the serial engine.
File path: `\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\qup-ssc-pinctrl.dtsi`. The default GPIO configuration can be overwritten as follows.

```
qup_ssc0_se2_i2c_sleep: qup_ssc0_se2_i2c_sleep{
    config = <&ssc_qupv3_se2_0 ssc_i2c_sleep_cfg>,
             <&ssc_qupv3_se2_1 ssc_i2c_sleep_cfg>;
};

qup_ssc0_se2_i3c_active: qup_ssc0_se2_i3c_active{
    config = <&ssc_qupv3_se2_0 ssc_i3c_active_sda_cfg>,
             <&ssc_qupv3_se2_1 ssc_i3c_active_scl_cfg>;
};
```

TLMM_MAP is a macro to initialize the active and sleep state GPIO configurations. For example, sample usage of the TLMM_MAP macro is as follows.

```
TLMM_MAP (active state pull type, drive strength, sleep state pull type)
```

Qualcomm TEE

The QUP v3 serial engine for I2C can be configured as follows.

File path: `\trustzone_images\core\settings\buses\i2c\qupv3\config\aspen\tz\i2c_devcfg_user.h`.

```
#define ENABLE_I2C_08
```

The `ENABLE_I2C_<num>` number is based on the serial number of the serial engine (starting from 0). For example, if there are two QUPs: QUPV3_0 with seven serial engines and QUPV3_1 with eight serial engines, then the user must enable QUPV3_2_SE2. The macro should be `ENABLE_I2C_08`.

GPIO configuration: The drive strength and pull are configured per PIN for SDA at zero index and SCL at one index. File path: `trustzone_images\core\settings\buses\i2c\qupv3\config\aspen\tz\i2c_devcfg_user.c`

```
ifdef ENABLE_I2C_00
i2c_plat_device_config_user i2c_device_user_config_qup1_se0 =
{
    {0,0},         //.drive_strength
       {3,3},        //.pull
    0xff,         //.gpii_idx (gpii assigned from tz ac)
    0,            //.mode_select  not supported for TZ
    0,            //.flags not supported for TZ
};
endif
```

QUPAC access control: It controls the firmware loading and access control permission from the Qualcomm TEE subsystem are configured in the QUPAC access file at `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`.

6.4 Share TOP QUP and SSC QUP using eGPIO

This section describes a shared use case where serial engines (SEs) are shared between the application processor subsystem (APPS) and the sensor subsystem controller (SSC) using extended GPIO (eGPIO).

The following table lists the SEs that can be shared between APPS and SSC using eGPIO.

Table 6-7 SSC LPI bus configurations

SSC GPIO	MSM GPIO	LPI serial engine	Supported protocols	MUX with TOP QUP
SSC_0-1	103-104	SSC_SE_0	I3C (/I2C)	–
SSC_2-3	105-106	SSC_SE_1	I2C (/I3C)	–
SSC_4-7	107-110	SSC_SE_2	▪ UART-4W (/SPI/I3C/I2C) ▪ SSC_SE2 supports 6 lanes ▪ SSC_4 supports 9 lanes	–

Table 6-7 SSC LPI bus configurations (cont.)

SSC GPIO	MSM GPIO	LPI serial engine	Supported protocols	MUX with TOP QUP
SSC_8-9	111-112	SSC_SE_3	I3C (/I2C)	Yes, TOP_SE_3
SSC_10-13	113-116	SSC_SE_4	■ 2 I/Os ■ Debug 2W UART (SPI/I2C)	–
SSC_14-15	117-118	SSC_SE_5	UART-2W (/I2C)	Yes, TOP_SE_6 (L2, L3)
SSC_18-19	121-122	SSC_SE_7	I2C (/UART)	–
SSC_20-21	123-124	SSC_SE_8	I2C (/I3C)	–
SSC_22-23, 16-17	125-126, 119-120	SSC_SE_9 (SE_6)	■ UART-4W (/I3C/ I2C) ■ SSC_SE6 muxed with SE9 GPIO_120, 121	–
SSC_24-25	72, 40	SSC_SE_10	I2C	Yes, TOP_SE_8
SSC_26-29	131-134	SSC_SE_11	SPI(/I2C)	Yes, TOP_SE_2
SSC_30-31	28-29	SSC_SE_12	I2C	Yes ■ TOP_SE_9 ■ TOP_SE_6 L0,L1
SSC_32-35	127-130	SSC_SE_13	SPI (/I2C)	Yes, TOP_SE_7

Table 6-8 RSB shared between APPS and M55 processor

Use case	Description
Interactive mode	RSB I2C is allocated to TOP QUP v3 SE8.
Ambient mode	RSB I2C is allocated to SSC QUPv3 SE10.
Control handoff	■ I2C is multiplexed over eGPIO. ■ When APPS hands over control to M55: □ APPS disables eGPIO mode via CSR in TLMM. □ M55, with SE10 pre-configured, takes over RSB
Interrupt handling	■ External interrupts are routed to both APPS and M55. ■ RSB interrupt to M55 is enabled only during handover.

Table 6-9 QUP and GPIO mapping

Signal	GPIO Number
SCL	GPIO[40]
SDA	GPIO[72]
RESET	GPIO[75]
RSB_INT	GPIO[102]

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungmul.yoon@lge.com

I2C pin control configuration

- File: `vienna-pinctrl.dtsi`
- Runtime control
  - `qcom,apps`: Control to APPS (using `get_sync()` during runtime resume)
  - `qcom,remote`: Control to SSC (using `put_sync()` during runtime suspend)
  - GPIO configuration

```
                                        qupv3_se8_i2c_pins:
qupv3_se8_i2c_pins {
                                          qupv3_se8_i2c_sda_active {
                                            mux { pins = "gpio72"; function
= "qup0_se8_l0"; };
                                            config { pins = "gpio72";
drive-strength = <2>; bias-pull-up; qcom,apps; };
                                          };
                                          qupv3_se8_i2c_scl_active {
                                            mux { pins = "gpio40"; function
= "qup0_se8_l1"; };
                                            config { pins = "gpio40";
drive-strength = <2>; bias-pull-up; qcom,apps; };
                                          };
                                          qupv3_se8_i2c_sleep {
                                            mux { pins = "gpio72",
"gpio40"; function = "gpio"; };
                                            config { pins = "gpio72",
"gpio40"; drive-strength = <2>; bias-pull-down; qcom,remote; };
                                          };
                                        };
```

  - Pin mapping

```
static const struct msm_pingroup vienna_groups[] = {
                                        [40] = PINGROUP(40, qup0_se8_l1,
NA, NA, NA, NA, NA, NA, NA, NA, NA, egpio, 0, -1),
                                        [72] = PINGROUP(72, qup0_se8_l0,
NA, NA, NA, NA, NA, NA, NA, NA, NA, egpio, 0, -1),
```

Offload flow

- On boot
  - APPS (via `SMC_Client` driver) configures GPIOs for **QUPv3-SE8** (active mode).

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:31 GMT hyungchul.won@lge.com

- RSB Offload Entry (to SSC)
  a. APPS sets RSB to power-down mode (`PD_CONFIG`) to unvote LDOs.
  b. APPS resets `EGPIO_ENABLE` (bit 12 of `TLMM_GPIO_CFG40` at `0xF128000`) using `put_sync()`.
  c. APPS configures eGPIO to sleep mode (LPI QUP: QUPv3-SE10).
  d. APPS notifies M55 of ambient entry.
  e. M55 takes over RSB and ensures LPI TLMM QUPv3-SE10 is functional.
- RSB offload exit (to APPS)
  a. M55 sets RSB to power-down mode (`PD_CONFIG`) to unvote LDOs.
  b. APPS sets `EGPIO_ENABLE` to `1` using `get_sync()`.
  c. APPS configures eGPIO to active mode (TOP QUP: QUPv3-SE8).

NOTE ■ This shared use case is verified only for the I2C interface.

6.5 Configure I2C interface

This section provides information about the I2C software driver kernel configuration and device tree node changes.

Linux

The following driver kernel configurations are required to support the I2C interface.

- Driver source: `drivers/i2c/busses/i2c-msm-geni.c`
- Kernel `defconfig` file path: `kernel_platform/common/arch/arm64/configs/defconfig`

The following kernel configurations are to be enabled.

- `CONFIG_QCOM_GENI_SE=y`
- `CONFIG_I2C_CHARDEV=m`
- `CONFIG_I2C_QCOM_GENI=m` to configure user space applications
- `CONFIG_QCOM_GPI_DMA=m` to enable GSI support

To enable the I2C DT node for validations, apply the following patch to the `/msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi` file.

```
diff --git a/arch/arm64/boot/dts/qcom/<chipset>.dtsi b/arch/arm64/boot/dts/
qcom/<chipset>.dtsi
index 8575f0b..cced7c0 100644
--- a/arch/arm64/boot/dts/qcom/<chipset>.dtsi
+++ b/arch/arm64/boot/dts/qcom/<chipset>.dtsi
@@ -6865,3 +6865,7 @@
 <GIC_PPI 10 IRQ_TYPE_LEVEL_LOW>;
 };
 };
+
```

```
+ &i2c1 {
+ status = "ok";
+};
```

NOTE You should compile the kernel configuration and device tree changes. After compilation, you can load the images to the device to verify the interface. For information about interface verification, see the Verify I2C interface section.

6.6 Verify I2C interface

This section describes the validation procedure and test results for the I2C drivers and the Qualcomm drivers.

Prerequisites:

Make the following changes.

```
diff --git a/drivers/i2c/Makefile b/drivers/i2c/Makefile
index 3f71ce4..48f468f 100644
--- a/drivers/i2c/Makefile
+++ b/drivers/i2c/Makefile
@@ -13,6 +13,7 @@
 obj-$(CONFIG_I2C_SMBUS)        += i2c-smbus.o
 obj-$(CONFIG_I2C_CHARDEV)      += i2c-dev.o
 obj-$(CONFIG_I2C_MUX)          += i2c-mux.o
+obj-y                  += i2c-dev.o
 obj-$(CONFIG_I2C_ATR)          += i2c-atr.o
 obj-y                  += algos/ busses/ muxes/
 obj-$(CONFIG_I2C_STUB)         += i2c-stub.o


--- a/configs/vienna_perf.bzl
+++ b/configs/vienna_perf.bzl
@@ -32,6 +32,7 @@
     "CONFIG_GIC_INTERRUPT_ROUTING": "m",
     "CONFIG_HWSPINLOCK_QCOM": "m",
     "CONFIG_I2C_MSM_GENI": "m",
+    "CONFIG_I2C_CHARDEV": "m",
     "CONFIG_INPUT_PM8941_PWRKEY": "m",
     "CONFIG_INTERCONNECT_CLK": "m",
     "CONFIG_INTERCONNECT_QCOM_BCM_VOTER": "m",
```

Linux

For upstream I2C kernel test applications, see https://cdn.kernel.org/pub/software/utils/i2c-tools/i2c-tools-4.3.tar.gz.

Qualcomm Confidential - May contain Trade Secrets 2026-05-1 02:44:2 GMT hyungchul.won@lge.com

To cross-compile tools, do the following:

1. Download `i2c-tool` from https://cdn.kernel.org/pub/software/utils/i2c-tools/i2c-tools-4.3.tar.gz.
2. Extract the tool from the downloaded `tar` file.

   ```
   tar -xzvf <i2c-tool-path>
   ```

3. Change the current directory to the `i2c-tool` path.

   ```
   cd <i2c-tool-path>
   ```

4. Install the tool.

   ```
   sudo apt-get install gcc-aarch64-linux-gnu
   ```

5. Set up the environment for cross-compilation.

   ```
   export CC=aarch64-linux-gnu-gcc
   ```

6. Compile the tool.

   ```
   make USE_STATIC_LIB=1
   ```

   The binary is generated at `<i2c-tool-path>/tools/`.

Verify I2C driver

Validate the driver by checking for `dev` node (`/dev/i2c-0` and `/dev/i2c-1`) in the ADB shell.

1. To verify the I2C driver, do the following:
   a. Open the ADB shell.
   b. Mount the file system.

      ```
      mount -o remount,rw /usr
      ```

   c. Transfer files using SCP or similar tools.

      For example, `scp i2cdetect root@10.92.162.185:/bin`

   d. Assign permission to execute.

      ```
      chmod 777 i2cdetect
      ```

2. Verify an I2C device with the `i2cdetect` tool. For example, `./i2cdetect -y -r <i2c_instance_num>`.

   ```
   /lib # ./i2cdetect -y -r 0
   ```

   The following output is displayed.

```
0 1 2 3 4 5 6 7 8 9 a b c d e f 00: -- -- -- -- -- -- -- -- 10: -- -- --
-- -- -- -- -- -- -- -- -- -- -- 20: -- -- -- -- -- -- -- -- --
-- -- 2b -- -- -- -- 30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
-- 40: -- -- -- -- 44 -- -- -- -- -- -- -- -- -- -- 50: -- -- -- --
-- -- -- -- -- -- -- -- -- -- -- 60: -- -- -- -- -- -- -- -- --
-- -- -- -- -- -- 70: -- -- -- -- -- -- -- -- /lib # exit
```

View I2C detect help

For more information about I2C detection, and how to use it, run the following commands.

```
/lib # ./i2cdetect --help
```

The following output is displayed.

Qualcomm Confidential - May Contain Trade Secrets 2020-05-18 02:41:21GMT hyungchul.won@lge.com

```
Error: Unsupported option "--help"! Usage: i2cdetect [-y] [-a] [-q|-r]
I2CBUS [FIRST LAST] i2cdetect -F I2CBUS i2cdetect -l I2CBUS is an integer
or an I2C bus name If provided, FIRST and LAST limit of the probing range.
```

Identify probed device in DUT

To identify the device probed from the DUT, run the following command.

```
/lib # ./i2cdump -r 0-0xff 0 0x2b b
```

Sample output:

```
WARNING! This program can confuse your I2C bus, cause data loss and worse!
will probe file /dev/i2c-0, address 0x2b, mode byte Probe range limited
to 0x00-0xff. Continue? [Y/n] Y 0 1 2 3 4 5 6 7 8 9 a b c d e f
0123456789abcdef 00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ 90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................ f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................
```

Read I2C data from device

To read I2C data from the device, run the following commands.

```
/lib # ./i2cget 0 0x2b 4 b
```

Sample output:

```
WARNING! This program can confuse your I2C bus, cause data loss and worse!
Will read from device file /dev/i2c-0, chip address 0x2b, data address
0x04, using read byte data. Continue? [Y/n] Y 0x00
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungwon@lge.com

6.7 Debug I2C issues

The section describes the default logging method of the I2C software driver to enable logging the I2C transfer failures.

Linux

The I2C driver logs are enabled through the kernel dynamic debugging method. Enable `CONFIG_DYNAMIC_DEBUG` in `<workspace_path_of_LINUX_kernel_image>/sources/kernel/kernel_platform/kernel/arch/arm64/configs/qcom_defconfig` to support the dynamic debugging of the kernel drivers.

To enable and view the I2C driver logs in the kernel logs (`dmesg`), run the following commands.

```
mount -t debugfs none /sys/kernel/debug
echo -n "file i2c-msm-geni.c +p" > /sys/kernel/debug/dynamic_debug/control
echo -n "file i2c-core-base.c +p" > /sys/kernel/debug/dynamic_debug/control
echo -n "file i2c-dev.c +p" > /sys/kernel/debug/dynamic_debug/control
echo -n "file i2c-mux.c +p" > /sys/kernel/debug/dynamic_debug/control
echo -n "file msm_gpi.c +p" > /sys/kernel/debug/dynamic_debug/control
```

To debug the error message: `[ 8.583248] geni_i2c a94000.i2c: Invalid proto 1` for driver protocol load failures, do the following.

1. Identify the board type.

   Example: QUPV3_1_SE5 board type details

   ```
   B -    461251 - CDT Version:3,Platform ID:34,Major ID:1,Minor
   ID:0,Subtype:2
   ```

2. From the kernel log, locate the platform ID and subtype related details at `trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.xml`.
3. Obtain the Qualcomm TEE `QUPAC_Access.c` file configurations.
4. Identify the configuration specific to the serial engine.
5. Locate the platform ID type within the `QUPAC_Access.xml` file.
6. Map this platform ID type to the corresponding `qupv3_perms` structure within the `QUPAC_Access.c` file.
7. Verify the protocol and mode configurations in the `QUPAC_Access.c` file.

6.8 I2C examples

For information about the upstream device tree reference, see the `msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-qupv3.dtsi` DTSI file.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.ryu@lge.com

7 I3C

The SW6100/SW6100P Improved interintegrated circuit (I3C) interface delivers data rates up to 12.5 MHz while maintaining backward compatibility with legacy I2C devices. You can enable dynamic addressing, in-band interrupts, and hot-join capabilities for sensor subsystem use cases. Configuration is available through aDSP/SLPI and M55 processor device tree settings.

The I3C interface has been developed to provide a fast, low-cost, low-power, 2-wire digital interface for connected I3C devices. The I3C interface is intended to improve upon the features of the I2C interface, preserving backward compatibility.

The 2-wire serial interface supports up to 12.5 MHz. Legacy I2C devices co-exist on the same bus. The I3C bus supports in-band interrupts, hot join, synchronous timing, and asynchronous time stamping. During the write operation, the transition bit is of 8-bit data parity. During the read operation, the controller uses the transition bit to either proceed with the next data or abort the read operation.

The data phase can be either push-pull or open-drain, based on the I3C capability of the device (for mixed bus, the I3C controller can address the I2C target).

- The address followed by the START condition is open-drain (arbitration phase).
- The address followed by a repeated START condition is push-pull.

The following figure illustrates the I3C hardware and software entities.

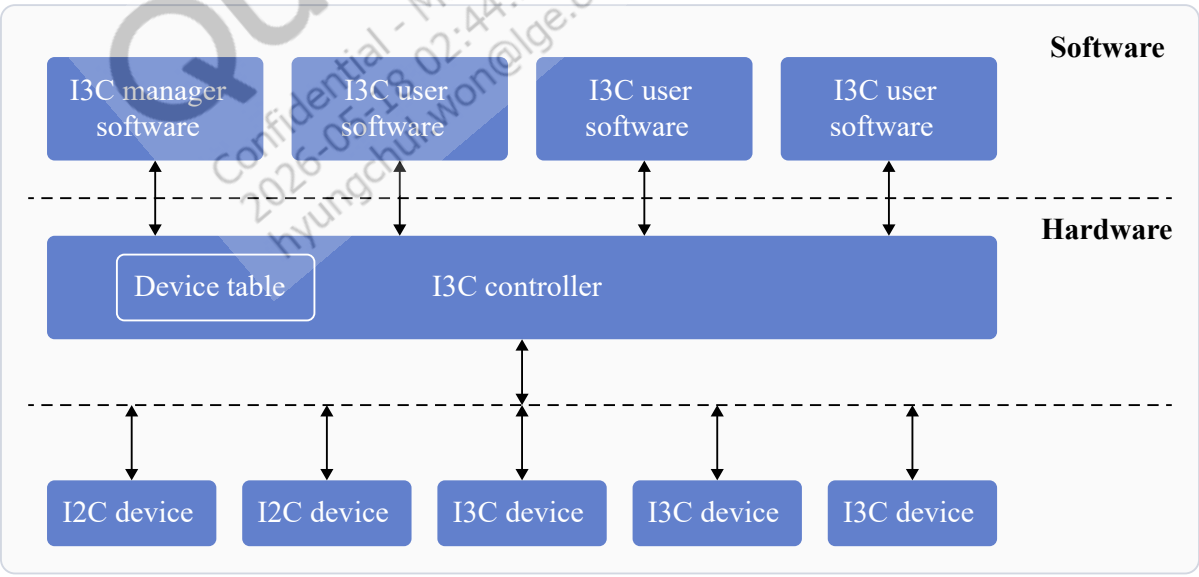


Figure 7-1 I3C block diagram

Qualcomm Confidential - May Contain Trade Secrets 2026-05-13 02:44:27 GMT hyungchulWon@lge.com

The key features of the I3C architecture are as follows:

- I3C controller driver software: software-privileged entity, capable of enumeration and bus configuration.
- I3C client software: software entities that use the I3C bus for read/write to devices.

I3C packet frame structure

A frame begins with START, command (8), data (8), transition (1), and STOP.

Figure 7-2 I3C packet frame

Where:

- ACK: Acknowledge (SDA low)
- S: Start condition
- Sr: Repeat start condition
- P: Stop condition
- T: Transition bit alternative to ACK/NACK

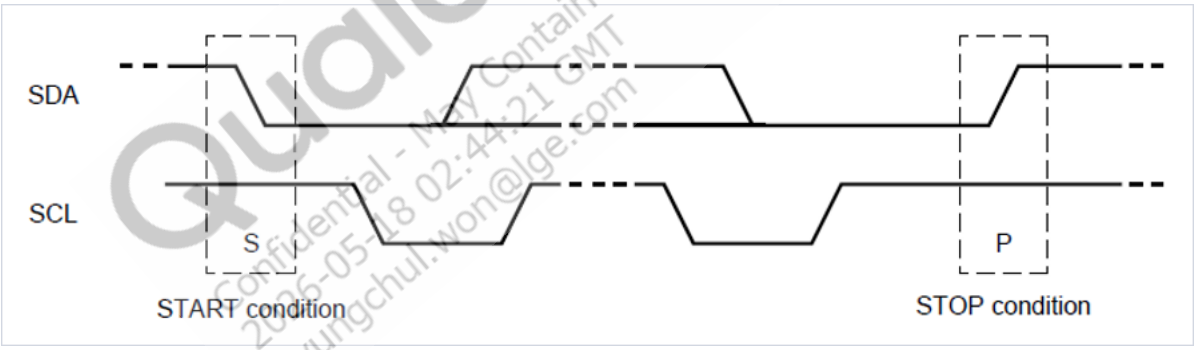


Figure 7-3 I3C sequence

I3C bus initialization

The I3C bus initialization procedure is as follows:

1. The I3C controller driver software initializes the I3C controller (firmware and configuration settings).
2. The I3C controller driver software reads the I3C configuration and the device database, including:
   - List of I2C static address devices.
   - List of I3C static address devices.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

- List of expected I3C dynamic devices: vendor ID, device ID, predefined dynamic address, and associated drivers.
- Optional hot-join allowed devices (vendor ID, device ID, and predefined address).

3. Using the I3C controller software, the driver enumerates the devices on the bus (set local addresses).
4. The I3C controller driver software writes an I3C controller with the required devices and the bus characteristic information:
   - Operation frequency
   - Pure/legacy I2C mode
   - SDR/HDR enable/disable (and types)
   - IBI capable devices and expected data bytes
5. The I3C client software stack receives the following information about the I3C configuration and device database.
   - List of I2C static address devices.
   - List of I3C static address devices.
   - List of expected I3C dynamic devices: vendor ID, device ID, predefined dynamic address, and associated drivers.
   - Optional hot-join allowed devices (vendor ID, device ID, and predefined address).

7.1 I3C features

The aDSP/SDC I3C subsystem supports the following features.

- Address phase at frequency 400 kHz.
- Data phase at frequency 12,500 kHz.
- Backward compatibility with legacy I2C.
- Dynamic addressing of targets.
- Single data rate (SDR) mode.
- CCC commands according to the MIPI I3C specification

7.2 I3C interface

I3C use cases in the aDSP/SLPI software support sensor device communication.

NOTE I3C doesn't support Linux use cases.

Qualcomm Confidential - May Contain Trade Secrets 2025-05-18 02:44:21 GMT hyungbui.kim@lge.com

The software driver and the configuration support for the I3C functionality for device communication are listed in the following table.

Table 7-1 I3C interface: aDSP/SLPI/SDC

File type	Description
QUP v3 serial engine configuration	adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\ssc-qupv3.dtsi
Firmware configuration settings	\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi
Public APIs	\adsp_proc\core\api\common\buses\i2c_api.h

Table 7-2 I3C interface: M55 processor

File type	Description
QUP v3 serial engine configuration	\adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi
Firmware configuration settings	ect.wasp.0.0\core.wasp\buses\i2c\zephyr\i3c_qcom.c

7.3 I3C software

The I3C firmware for the QUP v3 serial engine is loaded with SSC QUP during the bootup sequence. The configuration file is present in the aDSP build. The source code path to enable the I3C configuration for a specific QUP v3 serial engine instance is at \adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_fw\fw_devcfg.dtsi.

Sample of SSC QUP SE0 or SE1 loaded with the I3C firmware.

```
ssc_qup_0{
    se0_cfg {
        offset                  = /bits/ 32 <0x80000>;
        ibi_base                = /bits/ 32 <0x22100000>;
        protocol                = /bits/ 32 <SE_PROTOCOL_I3C>;
        se_island_config        = /bits/ 16 <SNS_ISLAND_TYPE>;
        tre_list_size           = /bits/ 8 <1>;
        ibi_se_index            = /bits/ 8 <0>;
        se_mode                 = /bits/ 8 <GSI>;
        load_fw                 = /bits/ 8 <1>;
        dfs_mode                = /bits/ 8 <1>;
    };
```

GPIO configuration: Each serial engine in the QUP common driver is configured with the default GPIO configuration. The QUP common driver selects the GPIO configuration according to the protocol loaded in the serial engine. The source code path for the GPIO configuration of a specific I3C serial engine instance is \adsp_proc\core\dt_settings\soc\aspen\core\buses\qup_common\qup-ssc-pinctrl.dtsi.

The default GPIO configuration can be overwritten as follows:

```
/*qup_ssc0_se0_pinctrl*/
    qup_ssc0_se0_i2c_active: qup_ssc0_se0_i2c_active{
        config = <&ssc_qupv3_se0_0 ssc_i2c_active_sda_cfg>,
                 <&ssc_qupv3_se0_1 ssc_i2c_active_scl_cfg>;
    };

    qup_ssc0_se0_i2c_sleep: qup_ssc0_se0_i2c_sleep{
        config = <&ssc_qupv3_se0_0 ssc_i2c_sleep_cfg>,
                 <&ssc_qupv3_se0_1 ssc_i2c_sleep_cfg>;
    };

    qup_ssc0_se0_i3c_active: qup_ssc0_se0_i3c_active{
        config = <&ssc_qupv3_se0_0 ssc_i3c_active_sda_cfg>,
                 <&ssc_qupv3_se0_1 ssc_i3c_active_scl_cfg>;
    };

    qup_ssc0_se0_i3c_sleep: qup_ssc0_se0_i3c_sleep{
        config = <&ssc_qupv3_se0_0 ssc_i3c_sleep_cfg>,
                 <&ssc_qupv3_se0_1 ssc_i3c_sleep_cfg>;
    };

    qup_ssc0_se0_i3c_ibi_active: qup_ssc0_se0_i3c_ibi_active{
        config = <&ssc_qupv3_se0_0 ssc_i3c_ibi_active_sda_cfg>,
                 <&ssc_qupv3_se0_1 ssc_i3c_ibi_active_scl_cfg>;
    };

    qup_ssc0_se0_i3c_ibi_sleep: qup_ssc0_se0_i3c_ibi_sleep{
        config = <&ssc_qupv3_se0_0 ssc_i3c_ibi_sleep_cfg>,
                 <&ssc_qupv3_se0_1 ssc_i3c_ibi_sleep_cfg>;
    };
```

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.woo@lge.com

8 USB

The SW6100/SW6100P Universal serial bus (USB) subsystem uses the Synopsys DesignWare USB 2.0 controller with high-speed (480 Mbps) peripheral mode and Type-C connector support via PWM6100. You can configure USB compositions for ADB, diagnostics, UVC, and UAC use cases, enable low-power mode for wearable applications, and troubleshoot enumeration issues using debugfs traces. Boot loader configuration through QDTE and kernel device tree settings control USB behavior.

USB is an industry standard that allows data exchange and delivery of power between various types of electronics. It can run at different speeds such as low speed at 1.5 Mbps, full speed at 12 Mbps, high speed at 480 Mbps.

Synopsys DesignWare® Core HighSpeed USB 2.0 powers the USB on the Qualcomm SoC and is connected to one eUSB PHY. This PHY is wired to the physical Type-C port and facilitates communication to the external world.

The following are the key hardware components of the USB.

- **USB controllers**
  - The primary controller is a Synopsys DesignWare Core HighSpeed USB 3.x controller (Gen1/ Gen2).
    - Synopsys PHY for high-speed USB.
- **Synopsys DesignWare core HighSpeed USB 2.x controller features**
  - Synopsys DesignWare Core HighSpeed USB 2.x controller can be configured as peripheral-only.
  - Supports all transfer types (control, bulk, interrupt, and isochronous).
  - Device mode supports HighSpeed (480 Mbps), and full-speed (12 Mbps) operations, and up to 16 bidirectional endpoints (including the control pipe `ep0`).
  - Link power management.
- **USB PHY access method**
  - Register-level interface through AHB2PHY for performing PHY-related operations.
- **USB Type-C**
  - Supports USB Type-C and power delivery using the PWM6100 controller.
  - Supports the PWM6100 software driver updates according to the UCSI framework after the delivery controller determines the Type-C orientation, role, and mode of the connected link partner.

Qualcomm Confidential - May Contain Trade Secrets 2026-03-18 02:44:21 GMT hyunchul.kim@lge.com

Clocks

The following tables list the clocks and operating frequencies required for the USB controller, and the high speed PHY to function.

Table 8-1 USB controller clocks

Clock name	Operating frequency	Description
gcc_usb30_prim_master_clk "core_clk"	66 MHz high speed	Asynchronous to the bus clock; clock rates defined within the device tree node
gcc_cfg_noc_usb3_prim_axi_clk "iface_clk"	66 MHz high speed	Auxiliary bus controller unit clock.
gcc_aggre_usb3_prim_axi_clk "bus_aggr_clk"	66 MHz high speed	Clock that feeds into the aggregator2 module, which controls data flow from the USB AXI to the NoC
gcc_usb30_prim_mock_utmi_clk "utmi_clk"	19.2 MHz	The internal controller ref_clk used for generating the ITP counter when the USB transceiver macrocell interface (UTMI)/UTMI+ Low Pin interface (ULPI) is suspended
gcc_usb30_prim_sleep_clk "sleep_clk"	32 kHz	Sleep clock
rpmh_cxo_clk "ref_clk_src"	19.2 MHz	Reference clock source to the HS-PHY
gcc_usb_phy_cfg_ahb2phy_clk "cfg_ahb_clk"	100 MHz	Clock required for the AHB2PHY block (frequency based off PNoC frequency).

Table 8-2 High-speed PHY clocks

Clock name	Operating Frequency	Description
rpmh_cxo_clk "ref_clk_src"	19.2 MHz	Reference clock source to the HS-PHY
gcc_usb_phy_cfg_ahb2phy_clk "cfg_ahb_clk"	100 MHz	Clock required for the AHB2PHY block (frequency based off PNoC frequency)

Voltage rails

The following table lists the required voltage rails for the HighSpeed PHY.

Table 8-3 USB voltage rails

Voltage rails	Voltage levels (max/nom/min)			Device mode	Description
VREG L26	1.2	1.1	0	HS-PHY	1.2 V regulator used for HS-PHY
VREG L25	0.8	0.07	0	HS-PHY	0.8 V regulator used for HS-PHY

Qualcomm Confidential – May Contain Trade Secrets 2025-05-18 02:44:21 GMT hyungchul.won@lge.com

Interrupts

The following table lists the various interrupts used by the USB controller to notify events.

Table 8-4 USB controller interrupts

Interrupt name	SW6100/ SW6100P interrupts	Interrupt events PDC wake-up kernel handling		Description
hs_phy_irq	68	D+ changes	–	Only used when the system is in VDD min/XO shutdown
pwr_event_irq	86	–	■ USB PHY power state changes ■ Exit/enter P3/L2	Used as the main controller wake-up handle when the system isn't in power collapse
core_irq	83	–	■ Bus events ▫ Suspend ▫ Resume ▫ Reset ■ Controller event completion ▫ Transfer completion ▫ Command completion	Main USB interrupt that manages all USB controller events

Interconnect

The following table lists the various interconnects used by the USB controller.

Table 8-5 USB interconnects

Interconnect name	Controller	Target	Interconnect path bandwidths in MBps
USB-DDR	MASTER_USB3_0	SLAVE_EBI1	■ USB_MEMORY_AVG_HS_BW MBps_to_icc(240) ■ USB_MEMORY_PEAK_HS_BW MBps_to_icc(700)
APPS-USB	MASTER_APPSS_PROC	SLAVE_USB3_0	■ APPS_USB_AVG_BW 0 ■ APPS_USB_PEAK_BW MBps_to_icc(40)

USB controller reset using clock control

The following table lists the reset methods used for the USB controller and PHY.

Table 8-6 USB clock reset methods

Clock name	Reset control	Description
GCC_QUSB2PHY_PRIM_BCR "phy_reset"	HS-PHY	Resets the HS-PHY
GCC_USB30_PRIM_BCR "core_reset"	USB controller	Clock controller output to reset the USB controller

Qualcomm Confidential - May Contain Trade Secrets 2026-05-19 02:44:21 GMT hyungchul.won@lge.com

USB controller software reset using register

The following table lists the register options to reset the USB controller.

Table 8-7 USB controller resets

USB controller register	USB register bit field	Reset control	Description
DWC3_DCTL	CSFTRST [Bit 30]	USB controller	Resets the USB controller device stack.
DWC3_GCTL	CORESOFTRESET [Bit 11]	-	Global reset for the DWC3 controller

USB controller and SoC integration

The intellectual property and PHYs of the USB controller are integrated into the SoC as shown in the following figure.

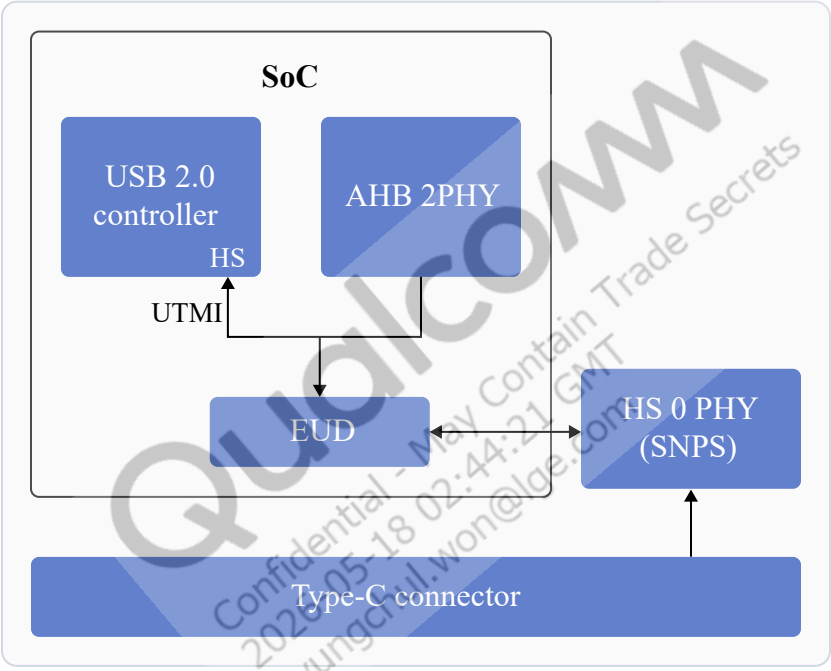


Figure 8-1 USB controller PHYs and SoC integration

The Synopsys DesignWare Core HighSpeed USB 2.0 intellectual property controls only the core functionality and not the device specifications, such as clocks, interconnects, regulators, and GDSCs. The Synopsys DesignWare Core intellectual property is embedded inside a `Qscratch` wrapper (intellectual property and software driver), which takes up the responsibility of managing the required resources (clocks, interconnects, interrupts, GDSC, and regulators) during the probe, suspends, or resume state. Both these drivers coexist to ensure the USB functionality.

For information about the USB `Qscratch` wrapper driver, see https://github.com/torvalds/linux/blob/master/drivers/usb/dwc3/dwc3-qcom.c. For information about the controller core driver, see https://github.com/torvalds/linux/blob/master/drivers/usb/dwc3/core.c.

USB Type-C connector system software interface (UCSI)

- The USB Type-C connector system software interface is available in the Linux kernel as a library. It defines a set of registers and data structures, which are used to interface with the USB Type-C connectors on a system. The USB Type-C connectors on the platform are referred to as the platform policy manager (PPM) and the system software component is referred to as the OS policy manager (OPM).
- Qualcomm reference design uses the UCSI `Glink` driver to manage the communication between the OPM on the application processor and the PPM, which is the charger firmware running on the remote subsystem (aDSP) over PMIC GLINK.
- Following are the references of the drivers involved:
  - `/kernel_platform/common/drivers/usb/typec/ucsi/ucsi.c`
  - `kernel_platform/soc-repo/drivers/usb/typec/ucsi/ucsi_qti_glink.c`
  - `kernel_platform/soc-repo/drivers/usb/dwc3/dwc3-msm-core.c`
  - `/kernel_platform/soc-repo/drivers/usb/phy/phy-msm-m31-eusb2.c`
  - `kernel_platform/soc-repo/drivers/soc/qcom/qti_pmic_glink.c`

8.1 USB features

Qualcomm chip hardware SoCs allow working in the peripheral/device mode only.

Table 8-8 USB features: Linux

Feature	Description
USB_DWC3 controller	Synopsys DesignWare Core is supported by default in the Linux kernel.
USB_DWC3_QCOM	`Qscratch` wrapper using the Synopsys DesignWare Core for the USB functionality.
Runtime power management (RPM)	Linux supports RPM with the software driver file `dwc3-msm-core.c`, for low-power mode operations.
Low-power mode (LPM)	LPM provides power-saving options from both HS-PHY as well as the controller.

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:41:21 GMT hyungchul.won@lge.com

8.2 USB architecture

The Qualcomm USB software architecture primarily consists of a downstream kernel driver.

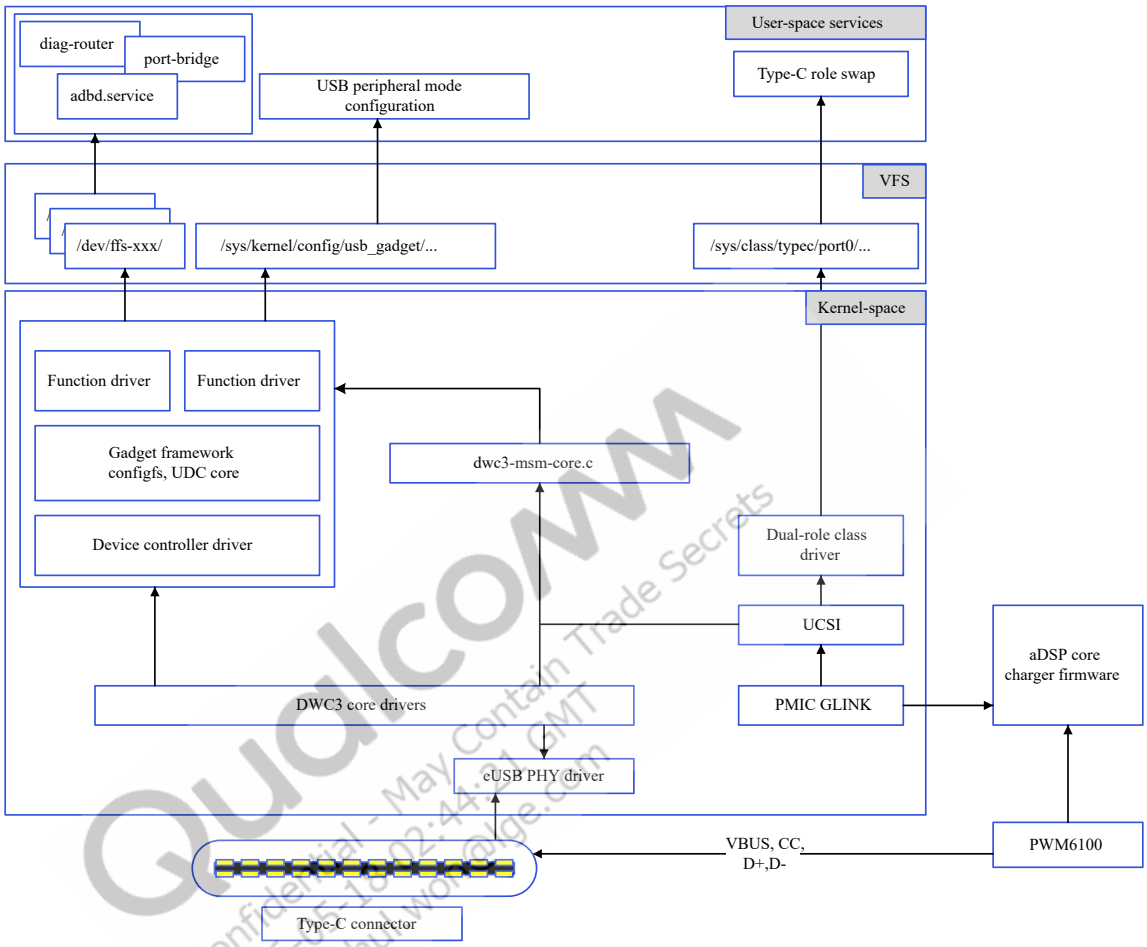


Figure 8-2 USB software architecture

The sandbox is implemented with the following Qualcomm-specific features.

- USB_DWC3_QCOM: The primary glue driver is responsible for the USB functionality on device mode, depending on the host PC connected. The device tree entry for the glue driver consists of resources, such as clocks, power rails, and interrupts. This feature uses the DWC3 core driver as a library, which controls the actual functionality of the DWC3 controller. For more details, see `kernel_platform/soc-repo/drivers/usb/dwc3/dwc3-msm-core.c`.
- Synopsys femto PHY: eUSB PHY is the high-speed USB PHY responsible for controlling `D+/D-` lines, and facilitates data transfers along with charger detection. For more details, see `kernel_platform/soc-repo/drivers/usb/phy/phy-msm-m31-eusb2.c`.

8.3 USB interfaces

USB supports multiple interfaces to transfer audio, video, debug information, and tethering. Each of the following USB interface communication protocol is different and has its own protocols in addition to the standard USB protocol.

- Android debug bridge (ADB)
  - The USB ADB is a debug interface, providing access to the system through the USB connection. For more information about ADB, see https://developer.android.com/tools/adb.
  - The filenames are the virtual `eps` names of the `dev` node. For more information about `f_fs.c`, see `kernel_platform/common/drivers/usb/gadget/function/f_fs.c`.
  - User space service: The user space daemon `adbd.service` operates ADB and is responsible for establishing the connection between the underlying kernel driver and the host PC.
- Diagnostics (diag)
  - Diag is a diagnostics framework used for collecting log data from various subsystems, and debugging. The diag data is transmitted over USB to the host PC.
  - Kernel driver: Diag uses `f_fs.c` to expose the `/dev/ffs-diag` node, which is operated by a user space service for various operations. The `dev` node has the following three files:
    - `ep0`: control operations
    - `ep1`: read operations
    - `ep2`: write operations

    These filenames are the virtual `eps` names of the `dev` node. For more information about `f_fs.c`, see `kernel_platform/common/drivers/usb/gadget/function/f_fs.c`.
- Dial-up networking (DUN)
  - DUN is a serial Interface for modem communication use case devices on dynamic plug and play I/O buses such as USB.
  - Kernel driver: DUN uses a driver `f_cdev.c`, which is present in the downstream kernel and directly communicates with the modem interfaces or stack
- Remote network driver interface specification (RMNET)
  - RMNET is a network device on the dynamic plug and play I/O bus (USB).
  - RMNET uses a `f_gsi.c` driver, which is present in the downstream Linux kernel and directly communicates with the network interfaces or stack.
- Remote network driver interface specification (RNDIS)
  - RNDIS is a specification developed by Microsoft for network devices on dynamic plug and play I/O buses such as USB.
  - Kernel driver: It uses a `f_rndis` driver, which is present in the upstream Linux kernel and directly communicates with the network interfaces or stack.

Qualcomm Confidential - May Contain Trade Secrets
2026-05-18 02:44:21 GMT
hyungchul.yoon@lge.com

8.4 USB software

The following USB features are supported in software.

Table 8-9 USB software features

Feature	Description
Peripheral ADB	ADB functionality is integrated over functionFS (FFS)
Peripheral diag	Diag functionality is integrated over FFS
Peripheral USB link power management	USB device mode LPM implementation
USB Type-C	Supported by PWM6100; supports current charging, CC logic, and HighSpeed, all performed in the embedded controller

8.5 USB tools

For more information about platform tools (USB/fastboot), see https://developer.android.com/tools/releases/platform-tools.

8.6 Configure USB boot loader

The device tree parameters for tuning the high-speed USB signal quality in the boot loader can be modified at`\boot_images\boot\QcomPkg\SocPkg\Aspen\Library\UsbSharedLib\UsbSharedLib.c`.

8.7 Customize USB device

This section describes the requirements for various configurations and customizations in the USB software.

Example shell script for a USB composition with diag and ADB interfaces

```
\vendor\qcom\opensource\usb\etc\init.qcom.usb.rc
\vendor\qcom\opensource\usb\etc\init.qcom.usb.sh


            "vienna")
            setprop persist.vendor.usb.config diag,qdss,rmnet,adb
            ;;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-10 02:44:21 GMT hyungshul.woo@lge.com

8.8 Verify USB device

The following table lists the various methods to verify the USB device and host modes.

Table 8-10 USB device and host mode verification

USB mode	Feature	Description
USB device mode	ADB device	Confirms the device enumerated on host PC side or check the device manager about USB port.
	USB LPM with cable connects and disconnects	Disconnects USB manually so that the `dwc3-qcom` module changes to low-power mode.

8.9 Debug USB issues

You can retrieve debugging logs using `regdumps`, debug `ftraces`, `configfs` nodes. The logs provide visibility into the event and controller state details when debugging issues with low-power mode entry-exit, SMMU faults, unclocked accesses.

Trace USB

The `debugfs` tracing provides a deeper view into each transaction over the USB line. To view the list of traces, run the following command.

```
ls /sys/kernel/debug/tracing/events/dwc3
```

NOTE Ensure that `debugfs` is mounted. If not mounted, run the following command to mount `debugfs`.

```
mount -t debugfs none /sys/kernel/debug
```

Following are the traces available for verifying data transfers in the gadget stack/USB Type-C connector system software interface (UCSI).

```
dwc3_alloc_request  dwc3_event              dwc3_gadget_generic_cmd  enable
dwc3_complete_trb   dwc3_free_request       dwc3_gadget_giveback     filter
dwc3_ctrl_req       dwc3_gadget_ep_cmd      dwc3_prepare_trb
dwc3_ep_dequeue     dwc3_gadget_ep_disable  dwc3_readl
dwc3_ep_queue       dwc3_gadget_ep_enable   dwc3_writel
```

To list the available events of the USB video class (UVC) gadget driver, run the following command.

```
ls /sys/kernel/debug/tracing/events/gadget
```

The following output is displayed.

```
enable                         usb_gadget_activate
  filter                         usb_gadget_clear_selfpowered
  usb_ep_alloc_request           usb_gadget_connect
  usb_ep_clear_halt              usb_gadget_deactivate
  usb_ep_dequeue                 usb_gadget_disconnect
  usb_ep_disable                 usb_gadget_frame_number
  usb_ep_enable                  usb_gadget_giveback_request
  usb_ep_fifo_flush              usb_gadget_set_remote_wakeup
  usb_ep_fifo_status             usb_gadget_set_selfpowered
  usb_ep_free_request            usb_gadget_vbus_connect
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-10 02:44:21 GMT hyungbed.won@lge.com

```
  usb_ep_queue                usb_gadget_vbus_disconnect
  usb_ep_set_halt             usb_gadget_vbus_draw
  usb_ep_set_maxpacket_limit  usb_gadget_wakeup
  usb_ep_set_wedge
```

To list the available events in the UCSI driver, run the following command.

```
ls /sys/kernel/debug/tracing/events/ucsi
```

The following output is displayed.

```
enable  ucsi_connector_change  ucsi_register_port  ucsi_run_command
filter  ucsi_register_altmode  ucsi_reset_ppm
```

USB regdump

The USB `debugfs` provides the following information.

- Mode of operation.

  ```
  cat /sys/kernel/debug/usb/a600000.dwc3/mode
  ```

  Sample output:

  ```
  device
  ```

- State and transfer ring buffer (TRB) queues to all endpoints in device mode.
- Current link state.

  ```
  cat /sys/kernel/debug/usb/a600000.dwc3/link_state
  ```

  Sample output:

  ```
  On
  ```

- List processor (LSP) dump.

  ```
  cat /sys/kernel/debug/usb/a600000.dwc3/lsp_dump
  ```

  Sample output:

  ```
  GDBGLSP[0] = 0x20000001
  GDBGLSP[1] = 0x00003c80
  GDBGLSP[2] = 0x38100000
  GDBGLSP[3] = 0x00401000
  GDBGLSP[4] = 0x24bf1000
  GDBGLSP[5] = 0x2e808000
  GDBGLSP[6] = 0x00002e80
  GDBGLSP[7] = 0xfc03c002
  GDBGLSP[8] = 0x0b803fff
  GDBGLSP[9] = 0x00000000
  GDBGLSP[10] = 0x00000478
  GDBGLSP[11] = 0x00000478
  GDBGLSP[12] = 0x00000478
  GDBGLSP[13] = 0x00000478
  GDBGLSP[14] = 0x00000478
  GDBGLSP[15] = 0x00000478
  ```

- List the contents.

To list the contents, run the following command.

```
ls /sys/kernel/debug/usb/a600000.usb
```

Qualcomm Confidential – May Contain Trade Secrets 2025-5-18 02:14:21 GMT sunil.won@lge.com

Sample output:

```
ep0in    ep11out  ep14in   ep1out  ep4in   ep6out  ep9in       regdump
ep0out   ep12in   ep14out  ep2in   ep4out  ep7in   ep9out      testmode
ep10in   ep12out  ep15in   ep2out  ep5in   ep7out  link_state
ep10out  ep13in   ep15out  ep3in   ep5out  ep8in   lsp_dump
ep11in   ep13out  ep1in    ep3out  ep6in   ep8out  mode
```

The `regdump` command provides the current state of the register space for the following registers:

- Device mode registers, such as DCTL, DSTS, and DCFG
- Global registers, such as GCTL and GSTS

```
cd /sys/kernel/debug/usb/a600000.usb
cat regdump
```

Sample output:

```
GSBUSCFG0 = 0x2222000e
GSBUSCFG1 = 0x00001700
GTXTHRCFG = 0x00000000
GRXTHRCFG = 0x00000000
GCTL = 0x00102000
GEVTEN = 0x00000000
GSTS = 0x7e800000
GUCTL1 = 0x850c1802
GSNPSID = 0x5533330a
GGPIO = 0x00000000
GUID = 0x00060c17
GUCTL = 0x0d00c010
GBUSERRADDR0 = 0x00000000
GBUSERRADDR1 = 0x00000000
GPRTBIMAP0 = 0x00000000
GPRTBIMAP1 = 0x00000000
GHWPARAMS0 = 0x4020400a
GHWPARAMS1 = 0x0122493b
GHWPARAMS2 = 0x300024d2
GHWPARAMS3 = 0x08210084
GHWPARAMS4 = 0x47822010
GHWPARAMS5 = 0x04204108
GHWPARAMS6 = 0x0be60020
GHWPARAMS7 = 0x00000000
GDBGFIFOSPACE = 0x000a0000
GDBGLTSSM = 0x00000600
GDBGBMU = 0x20300000
GPRTBIMAP_HS0 = 0x00000000
GPRTBIMAP_HS1 = 0x00000000
GPRTBIMAP_FS0 = 0x00000000
GPRTBIMAP_FS1 = 0x00000000
GUCTL2 = 0x0000440d
VER_NUMBER = 0x00000000
VER_TYPE = 0x00000000
GUSB2PHYCFG(0) = 0x00002440
GUSB2PHYCFG(1) = 0x00000000
GUSB2PHYCFG(2) = 0x00000000
GUSB2PHYCFG(3) = 0x00000000
GUSB2PHYCFG(4) = 0x00000000
GUSB2PHYCFG(5) = 0x00000000
GUSB2PHYCFG(6) = 0x00000000
```

```
GUSB2PHYCFG(7) = 0x00000000
GUSB2PHYCFG(8) = 0x00000000
GUSB2PHYCFG(9) = 0x00000000
GUSB2PHYCFG(10) = 0x00000000
GUSB2PHYCFG(11) = 0x00000000
GUSB2PHYCFG(12) = 0x00000000
GUSB2PHYCFG(13) = 0x00000000
GUSB2PHYCFG(14) = 0x00000000
GUSB2PHYCFG(15) = 0x00000000
GUSB2I2CCTL(0) = 0x00000000
GUSB2I2CCTL(1) = 0x00000000
GUSB2I2CCTL(2) = 0x00000000
GUSB2I2CCTL(3) = 0x00000000
GUSB2I2CCTL(4) = 0x00000000
GUSB2I2CCTL(5) = 0x00000000
GUSB2I2CCTL(6) = 0x00000000
GUSB2I2CCTL(7) = 0x00000000
GUSB2I2CCTL(8) = 0x00000000
GUSB2I2CCTL(9) = 0x00000000
GUSB2I2CCTL(10) = 0x00000000
GUSB2I2CCTL(11) = 0x00000000
GUSB2I2CCTL(12) = 0x00000000
GUSB2I2CCTL(13) = 0x00000000
GUSB2I2CCTL(14) = 0x00000000
GUSB2I2CCTL(15) = 0x00000000
GUSB2PHYACC(0) = 0x00000000
GUSB2PHYACC(1) = 0x00000000
GUSB2PHYACC(2) = 0x00000000
GUSB2PHYACC(3) = 0x00000000
GUSB2PHYACC(4) = 0x00000000
GUSB2PHYACC(5) = 0x00000000
GUSB2PHYACC(6) = 0x00000000
GUSB2PHYACC(7) = 0x00000000
GUSB2PHYACC(8) = 0x00000000
GUSB2PHYACC(9) = 0x00000000
GUSB2PHYACC(10) = 0x00000000
GUSB2PHYACC(11) = 0x00000000
GUSB2PHYACC(12) = 0x00000000
GUSB2PHYACC(13) = 0x00000000
GUSB2PHYACC(14) = 0x00000000
GUSB2PHYACC(15) = 0x00000000
GUSB3PIPECTL(0) = 0x010e0002
GUSB3PIPECTL(1) = 0x00000000
GUSB3PIPECTL(2) = 0x00000000
GUSB3PIPECTL(3) = 0x00000000
GUSB3PIPECTL(4) = 0x00000000
GUSB3PIPECTL(5) = 0x00000000
GUSB3PIPECTL(6) = 0x00000000
GUSB3PIPECTL(7) = 0x00000000
GUSB3PIPECTL(8) = 0x00000000
GUSB3PIPECTL(9) = 0x00000000
GUSB3PIPECTL(10) = 0x00000000
GUSB3PIPECTL(11) = 0x00000000
GUSB3PIPECTL(12) = 0x00000000
GUSB3PIPECTL(13) = 0x00000000
GUSB3PIPECTL(14) = 0x00000000
GUSB3PIPECTL(15) = 0x00000000
GTXFIFOSIZ(0) = 0x06c7000a
GTXFIFOSIZ(1) = 0x06d10103
GTXFIFOSIZ(2) = 0x07d40103
GTXFIFOSIZ(3) = 0x08d70103
```

Confidential – May Contain Trade Secrets
2020-05-18 02:44:21 GMT
hyungchul.kwon@lge.com

```
GTXFIFOSIZ(4) = 0x09da0083
GTXFIFOSIZ(5) = 0x0a5d0083
GTXFIFOSIZ(6) = 0x0ae00083
GTXFIFOSIZ(7) = 0x0b630083
GTXFIFOSIZ(8) = 0x00000000
GTXFIFOSIZ(9) = 0x00000000
GTXFIFOSIZ(10) = 0x00000000
GTXFIFOSIZ(11) = 0x00000000
GTXFIFOSIZ(12) = 0x00000000
GTXFIFOSIZ(13) = 0x00000000
GTXFIFOSIZ(14) = 0x00000000
GTXFIFOSIZ(15) = 0x00000000
GTXFIFOSIZ(16) = 0x00000000
GTXFIFOSIZ(17) = 0x00000000
GTXFIFOSIZ(18) = 0x00000000
GTXFIFOSIZ(19) = 0x00000000
GTXFIFOSIZ(20) = 0x00000000
GTXFIFOSIZ(21) = 0x00000000
GTXFIFOSIZ(22) = 0x00000000
GTXFIFOSIZ(23) = 0x00000000
GTXFIFOSIZ(24) = 0x00000000
GTXFIFOSIZ(25) = 0x00000000
GTXFIFOSIZ(26) = 0x00000000
GTXFIFOSIZ(27) = 0x00000000
GTXFIFOSIZ(28) = 0x00000000
GTXFIFOSIZ(29) = 0x00000000
GTXFIFOSIZ(30) = 0x00000000
GTXFIFOSIZ(31) = 0x00000000
GRXFIFOSIZ(0) = 0x03c20305
GRXFIFOSIZ(1) = 0x06c70000
GRXFIFOSIZ(2) = 0x06c70000
GRXFIFOSIZ(3) = 0x00000000
GRXFIFOSIZ(4) = 0x00000000
GRXFIFOSIZ(5) = 0x00000000
GRXFIFOSIZ(6) = 0x00000000
GRXFIFOSIZ(7) = 0x00000000
GRXFIFOSIZ(8) = 0x00000000
GRXFIFOSIZ(9) = 0x00000000
GRXFIFOSIZ(10) = 0x00000000
GRXFIFOSIZ(11) = 0x00000000
GRXFIFOSIZ(12) = 0x00000000
GRXFIFOSIZ(13) = 0x00000000
GRXFIFOSIZ(14) = 0x00000000
GRXFIFOSIZ(15) = 0x00000000
GRXFIFOSIZ(16) = 0x00000000
GRXFIFOSIZ(17) = 0x00000000
GRXFIFOSIZ(18) = 0x00000000
GRXFIFOSIZ(19) = 0x00000000
GRXFIFOSIZ(20) = 0x00000000
GRXFIFOSIZ(21) = 0x00000000
GRXFIFOSIZ(22) = 0x00000000
GRXFIFOSIZ(23) = 0x00000000
GRXFIFOSIZ(24) = 0x00000000
GRXFIFOSIZ(25) = 0x00000000
GRXFIFOSIZ(26) = 0x00000000
GRXFIFOSIZ(27) = 0x00000000
GRXFIFOSIZ(28) = 0x00000000
GRXFIFOSIZ(29) = 0x00000000
GRXFIFOSIZ(30) = 0x00000000
GRXFIFOSIZ(31) = 0x00000000
GEVNTADRLO(0) = 0xd6763000
```

```
GEVNTADRHI(0) = 0x00000008
GEVNTSIZ(0) = 0x00001000
GEVNTCOUNT(0) = 0x00000000
GHWPARAMS8 = 0x000004d0
GUCTL3 = 0x00010000
GFLADJ = 0x8c80c8a0
DCFG = 0x00a00988
DCTL = 0x80f00000
DEVTEN = 0x00000257
DSTS = 0x0082ed78
DGCMDPAR = 0x00000000
DGCMD = 0x00000000
DALEPENA = 0x00002bff
DEPCMDPAR2(0) = 0x00000000
DEPCMDPAR1(0) = 0xf7049000
DEPCMDPAR0(0) = 0x00000008
DEPCMD(0) = 0x00000006
DEPCMDPAR2(1) = 0x00000000
DEPCMDPAR1(1) = 0xf7049000
DEPCMDPAR0(1) = 0x00000008
DEPCMD(1) = 0x00010006
DEPCMDPAR2(2) = 0x00000000
DEPCMDPAR1(2) = 0xec0a50c0
DEPCMDPAR0(2) = 0x00000008
DEPCMD(2) = 0x00020007
DEPCMDPAR2(3) = 0x00000000
DEPCMDPAR1(3) = 0xd346d000
DEPCMDPAR0(3) = 0x00000008
DEPCMD(3) = 0x00030007
DEPCMDPAR2(4) = 0x00000000
DEPCMDPAR1(4) = 0xe9de6000
DEPCMDPAR0(4) = 0x00000008
DEPCMD(4) = 0x00060006
DEPCMDPAR2(5) = 0x00000000
DEPCMDPAR1(5) = 0xe19c9000
DEPCMDPAR0(5) = 0x00000008
DEPCMD(5) = 0x00040006
DEPCMDPAR2(6) = 0x00000000
DEPCMDPAR1(6) = 0xe19c8000
DEPCMDPAR0(6) = 0x00000008
DEPCMD(6) = 0x00070007
DEPCMDPAR2(7) = 0x00000000
DEPCMDPAR1(7) = 0xe19e1000
DEPCMDPAR0(7) = 0x00000008
DEPCMD(7) = 0x00050006
DEPCMDPAR2(8) = 0x00000000
DEPCMDPAR1(8) = 0x00000000
DEPCMDPAR0(8) = 0x00000000
DEPCMD(8) = 0x00000005
DEPCMDPAR2(9) = 0x00000000
DEPCMDPAR1(9) = 0xf3d08000
DEPCMDPAR0(9) = 0x00000008
DEPCMD(9) = 0x00080007
DEPCMDPAR2(10) = 0x00000000
DEPCMDPAR1(10) = 0x00000000
DEPCMDPAR0(10) = 0x00000000
DEPCMD(10) = 0x00000000
DEPCMDPAR2(11) = 0x00000000
DEPCMDPAR1(11) = 0x00000000
DEPCMDPAR0(11) = 0x00000000
DEPCMD(11) = 0x00000005
```

```
DEPCMDPAR2(12) = 0x00000000
DEPCMDPAR1(12) = 0x00000000
DEPCMDPAR0(12) = 0x00000000
DEPCMD(12) = 0x00000000
DEPCMDPAR2(13) = 0x00000000
DEPCMDPAR1(13) = 0xe7dcb330
DEPCMDPAR0(13) = 0x00000008
DEPCMD(13) = 0x00090007
DEPCMDPAR2(14) = 0x00000000
DEPCMDPAR1(14) = 0x00000000
DEPCMDPAR0(14) = 0x00000000
DEPCMD(14) = 0x00000000
DEPCMDPAR2(15) = 0x00000000
DEPCMDPAR1(15) = 0x00000000
DEPCMDPAR0(15) = 0x00000000
DEPCMD(15) = 0x00000000
DEPCMDPAR2(16) = 0x00000000
DEPCMDPAR1(16) = 0x00000000
DEPCMDPAR0(16) = 0x00000000
DEPCMD(16) = 0x00000000
DEPCMDPAR2(17) = 0x00000000
DEPCMDPAR1(17) = 0x00000000
DEPCMDPAR0(17) = 0x00000000
DEPCMD(17) = 0x00000000
DEPCMDPAR2(18) = 0x00000000
DEPCMDPAR1(18) = 0x00000000
DEPCMDPAR0(18) = 0x00000000
DEPCMD(18) = 0x00000000
DEPCMDPAR2(19) = 0x00000000
DEPCMDPAR1(19) = 0x00000000
DEPCMDPAR0(19) = 0x00000000
DEPCMD(19) = 0x00000000
DEPCMDPAR2(20) = 0x00000000
DEPCMDPAR1(20) = 0x00000000
DEPCMDPAR0(20) = 0x00000000
DEPCMD(20) = 0x00000000
DEPCMDPAR2(21) = 0x00000000
DEPCMDPAR1(21) = 0x00000000
DEPCMDPAR0(21) = 0x00000000
DEPCMD(21) = 0x00000000
DEPCMDPAR2(22) = 0x00000000
DEPCMDPAR1(22) = 0x00000000
DEPCMDPAR0(22) = 0x00000000
DEPCMD(22) = 0x00000000
DEPCMDPAR2(23) = 0x00000000
DEPCMDPAR1(23) = 0x00000000
DEPCMDPAR0(23) = 0x00000000
DEPCMD(23) = 0x00000000
DEPCMDPAR2(24) = 0x00000000
DEPCMDPAR1(24) = 0x00000000
DEPCMDPAR0(24) = 0x00000000
DEPCMD(24) = 0x00000000
DEPCMDPAR2(25) = 0x00000000
DEPCMDPAR1(25) = 0x00000000
DEPCMDPAR0(25) = 0x00000000
DEPCMD(25) = 0x00000000
DEPCMDPAR2(26) = 0x00000000
DEPCMDPAR1(26) = 0x00000000
DEPCMDPAR0(26) = 0x00000000
DEPCMD(26) = 0x00000000
DEPCMDPAR2(27) = 0x00000000
```

Qualcomm Confidential - May Contain Trade Secrets
2020-05-18 02:44:21 GMT
jwongchul.won@lge.com

```
DEPCMDPAR1(27) = 0x00000000
DEPCMDPAR0(27) = 0x00000000
DEPCMD(27) = 0x00000000
DEPCMDPAR2(28) = 0x00000000
DEPCMDPAR1(28) = 0x00000000
DEPCMDPAR0(28) = 0x00000000
DEPCMD(28) = 0x00000000
DEPCMDPAR2(29) = 0x00000000
DEPCMDPAR1(29) = 0x00000000
DEPCMDPAR0(29) = 0x00000000
DEPCMD(29) = 0x00000000
DEPCMDPAR2(30) = 0x00000000
DEPCMDPAR1(30) = 0x00000000
DEPCMDPAR0(30) = 0x00000000
DEPCMD(30) = 0x00000000
DEPCMDPAR2(31) = 0x00000000
DEPCMDPAR1(31) = 0x00000000
DEPCMDPAR0(31) = 0x00000000
DEPCMD(31) = 0x00000000
OCFG = 0x00000000
OCTL = 0x00000000
OEVT = 0x00000000
OEVTEN = 0x00000000
OSTS = 0x00000000
```

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

9 References

You can use the reference material to quickly look up QUPv3 concepts, transfer modes, access control, and firmware status checks. A glossary and supporting information help you interpret terms and configuration details as you work with SW6100/SW6100P interfaces.

9.1 QUP v3 overview

The QUP v3 is a highly flexible and programmable hardware for supporting a wide range of serial interface. A single QUP v3 serial engine hardware core provides up to eight serial interfaces. The two QUP v3 hardware cores are as follows:

- 11-serial engine core
- SSC QUP v3 hardware core with 14 serial engines available in the SSC_I/Os

The QUP v3 supports access from multiple hardware entities in the system. Each entity has its own execution environment (EE), a separate address space, and an interrupt line. For information about the various transfer modes that can be configured in QUP v3, see Supported transfer modes in QUP v3.

The following figure shows one GSI core/engine connected with up to eight serial engines (SE). You have the flexibility to customize configurations depending on the use case or the protocol of the serial engine. For information about how to customize configurations, see QUP v3 access control customization. To verify if the QUP v3 firmware is correctly flashed, see QUP v3 firmware status verification.

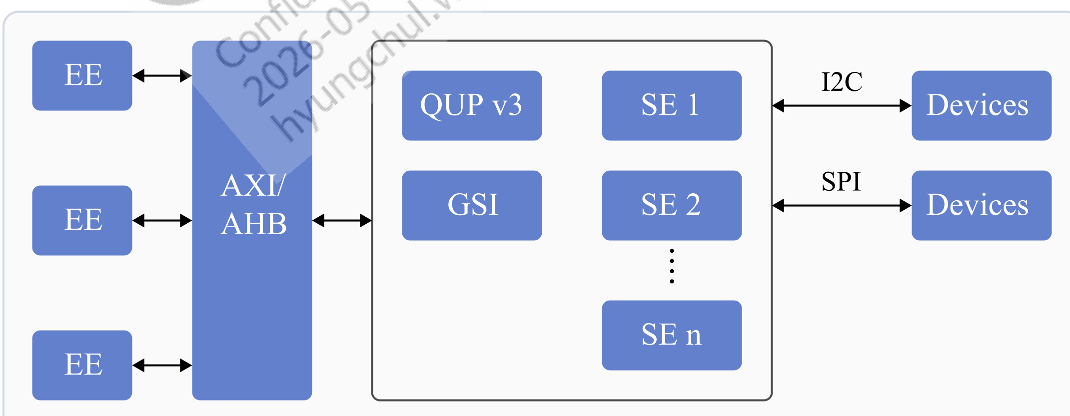


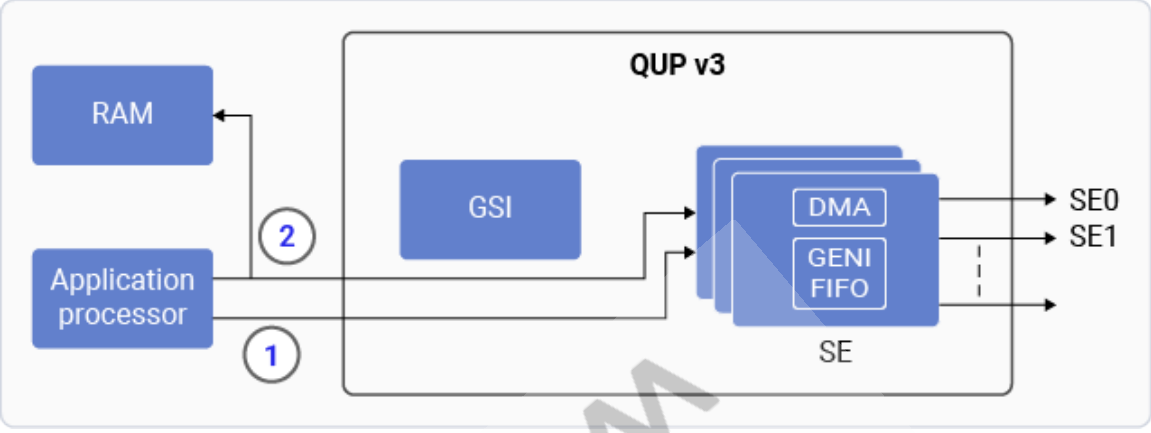
Figure 9-1 QUP v3 block diagram

Qualcomm Confidential – May Contain Trade Secrets
2026-05-18 07:44:21 GMT
hyungchul.won@lge.com

9.2 Supported transfer modes in QUP v3

The following modes can be configured in the QUP v3 serial engine.

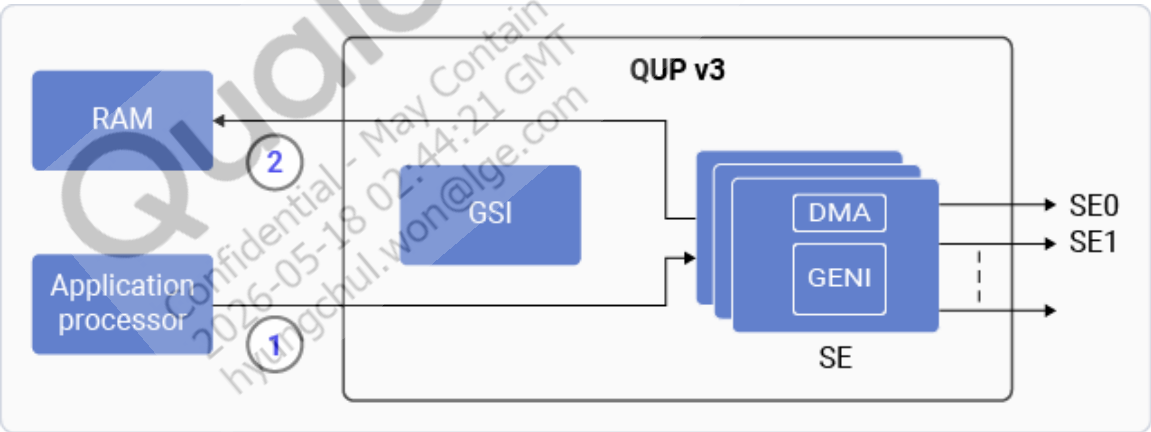
- **FIFO mode**



a. ①: The application processor configures the generic interface (GENI).

b. ②: The application processor processes Rx/Tx data. The data is transferred between GENI and memory.

- **DMA mode**

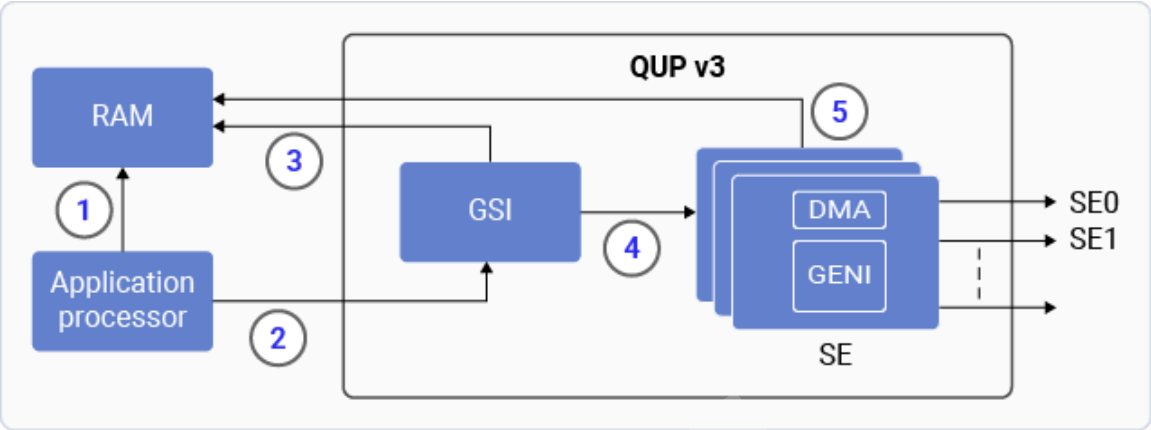


a. ①: The application processor initializes DMA.

b. ②: The serial engine configures GENI, and initiates the transfer process.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:21 GMT hyungchul.won@lge.com

- **GSI mode**



a. ①: The application processor prepares TRE.
b. ②: The application processor informs QUP (GSI).
c. ③: The GSI processes TRE.
d. ④: After the TRE completes processing, it's sent to GENI.
e. ⑤: The serial engine processes the Rx/Tx data.

NOTE The QUP v3 UART serial engine doesn't support the GSI mode.

9.3 QUP v3 access control customization

The QUP v3 user access file `QUPAC_Access.c` specifies the owners of the serial engine resource. Initially, it's populated according to the system I/O GPIO allocation. All serial engines must be listed to access the subsystem. It's flexible enough to list only the available serial engine on a particular device.

To customize the access control for the required serial engine protocol, configure the parameters in the `QUPAC_Access.c` file. The Qualcomm TEE image for the `QUPAC_Access.c` file is at `\trustzone_images\core\settings\buses\qup_accesscontrol\qupv3\config\aspen\QUPAC_Access.c`.

To specify the owner of the serial engine resource, modify the `QUPAC_Access.c` file to suit the board design.

The following use case specifies the default protocol that operates on an enabled serial engine. You can modify the code according to your board design.

Nonsecure mode use case in QUP v3 serial engine

The following nonsecure mode use cases are supported in the QUP v3 serial engine.

```
const QUPv3_se_security_permissions_type qupv3_perms_wdp[] =
{
  /*   PeriphID,          ProtocolID,               Mode,  NsOwner,
bAllowFifo, bLoad, bModExcl  */
  { QUPV3_1_SE0, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_TZ,
FALSE,    TRUE,  TRUE  }, // NFC ESE SPI
  { QUPV3_1_SE1, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,
```

```
FALSE,    TRUE,  FALSE }, // NFC Ctrl I2C
  { QUPV3_1_SE2, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // QCC UWB SPI eCoRRAL_SKU1 (SKU1), WDP
  { QUPV3_1_SE3, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // QCC UWB SPI
  { QUPV3_1_SE4, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_FIFO, AC_HLOS,
TRUE,     FALSE, FALSE }, // DEBUG_UART
  { QUPV3_1_SE5, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_FIFO, AC_HLOS,      TRUE,
   TRUE,  FALSE }, // APPS I2C
  { QUPV3_1_SE6, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE },
  { QUPV3_1_SE7, QUPV3_PROTOCOL_SPI,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE }, // eGPIO Touch SPI
// The dedicated SOCCP & DCP ownership usecase shall be taken care in XBL. No
need to mention the configs here
  /*{ QUPV3_1_SE8, QUPV3_PROTOCOL_UART_2W, QUPV3_MODE_GSI,  AC_DCP,
FALSE,    TRUE,  FALSE },*/ // DCP_UART
  { QUPV3_1_SE9, QUPV3_PROTOCOL_I2C,     QUPV3_MODE_GSI,  AC_HLOS,
FALSE,    TRUE,  FALSE },//touch
  /*QUPV3_1_SE10*/

};
```

QUP v3 serial engine access list description

The following variables are passed into the `QUPv3_se_security_permissions_type` structure.

Table 9-1 Security permission variables for QUP v3

Variables	Description
`PeriphID`	Serial engine peripheral to configure and assign.
`ProtocolID`	Macro of the required protocol.
`Mode`	Macro of FIFO/GSI/DMA modes.
`NsOwner`	Holds a macro of the image that needs access.
`bAllowFifo`	The boolean flag is set to `True` if the mode is `FIFO`, else the flag is set to `False`.
`bLoad`	The boolean flag value is set to `True` to load the protocol firmware.
`bModExcl`	This flag is exclusively for Qualcomm TEE. It's set to `True` when `NsOwner` is AC_TZ.

For more information about this macro, see `settings/buses/qup_accesscontrol/qupv3/interface/QupACCommonIds.h`.

Qualcomm Confidential - May Contain Trade Secrets 2026-05-18 02:44:27 GMT hyungchul.won@lge.com

9.4 QUP v3 firmware status verification

For serial engines to work, the QUP firmware must be flashed correctly. The firmware is delivered through the metabuild at `common/core_qupv3fw/aspen/qupv3fw.elf`. You can verify the firmware status by checking `GENI_FW_REVISION_RO (0xa8c068)`. For example, identify the register in the kernel log for the `0000ffff` value. In the following log, the `0000ffff` error indicates that the firmware isn't flashed correctly.

```
0a8c068: 0000ffff //Invalid firmware or firmware not loaded
00a80068: 00000126 //SPI
00a88068: 00000338 //I2C
```

Modify the `QUPAC_Access.c` file configuration only if you intend to use a protocol different from the default configuration.

The following sample log is displayed when configurations don't match after loading.

```
msm_geni_serial 898000.qcom,qup_uart:msm_geni_serial_startup: Invalid FW
                    255 loaded
```

9.5 Related documents

Title	Resource
Qualcomm Technologies, Inc.	
SW6100/SW6100P Device Tree Customization Guide - Linux Embedded	80-93264-19

9.6 Acronyms and terms

Acronym or term	Definition
aDSP	Application digital signal processor
ADB	Android debug bridge
AM	Ambient mode microcontroller
ASPM	Active state power management
AWM	Ambient mode wearable microcontroller
BDF	Bus device function
CAN	Controller area network
CRC	Cyclic redundancy check
CTS	Clear to send
DLLP	Data link layer packet
DTS	Device tree source
ECRC	End-to-end CRC
EE	Execution environment
ESE	Execute secure environment
FP	Fingerprint

Qualcomm Confidential - May Contain Trade Secrets 2026-05-16 02:44:21 GMT hyungpaul.yoo@lge.com

Acronym or term	Definition
GPIO	General-purpose input/output
GSI	Generic software interface
HID	Human interface device
I/O	Input/output
IOMMU	Input/output memory management unit
I2C	Interintegrated circuit
I3C	Improved interintegrated circuit
LSP	List processor
MS	Mass storage
MSI	Message signaled interrupt
NCM	Network control modem
NFC	Near-field communication
NIC	Network interface card
QoS	Quality of service
QIM	Qualcomm intelligent multimedia
RTS	Request to send
SCL	Serial clock line
SDK	Software development kit
SDL	Serial data line
SE	Serial engine
SLPI	Sensor low-power island
SPI	Serial peripheral interface
SPMI	System power management interface
SR-IOV	Single root I/O virtualization
SSC	Snapdragon sensor core
SWM	Sensor wearable microcontroller
TC	Traffic class
TEE	Trusted execution environment
TLP	Transaction layer packet
UEFI	Unified extensible firmware interface
UART	Universal asynchronous receiver-transmitter
UCSI	USB Type-C connector system software interface
V4L2	Video4Linux2
VC	Virtual channel
Yavta	Yet another V4L2 test application

Qualcomm Confidential – May Contain Trade Secrets 2026-05-18 02:41:21 GMT hyungchul.won@lge.com

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-16 02:44:21 GMT
hyungchul.won@lge.com